

Universidade da Beira Interior
Department of Computer Science



Kaonashi - Rust Rollups

By:

Beatriz Morais Laranjinha

Advisors:

Professor Doctor Paul Andrew Crocker

Professor Luís Pedro Arrojado da Horta

July 5, 2026

Acknowledgements

I would like to thank Professor Doctor Paul Andrew Crocker for his guidance throughout this project, for patiently answering my endless daily questions (and I truly mean endless, I do not think I let him have a moment of peace), and for the genuine interest he showed in this work, which in turn fueled my own. I would also like to thank Professor Luís Pedro Arrojado da Horta, who, despite his limited availability, always found time to care about this project and support its progress, and for everything he taught me throughout the course, having had the pleasure of being his student in several classes.

A special thanks to my parents, for their constant support and encouragement throughout my years at university. They made this journey easier and always pushed me to pursue work that truly fulfills me.

To my boyfriend, who inspires me, never lets me give up, and constantly makes me more curious to learn more, thank you for your patience through every meltdown along the way.

Contents

Contents	iii
List of Figures	vii
Acronyms	xi
Glossary	xiii
1 Introduction	1
1.1 Context	1
1.2 Motivation	1
1.3 Objectives	1
1.3.1 Why Kaonashi?	2
1.4 Challenges and Contributions	2
1.5 Report Outline	2
2 State of the Art	3
2.1 Introduction	3
2.2 Blockchain	3
2.3 Bitcoin	4
2.4 Solana	4
2.4.1 Proof of Work	4
2.4.2 Proof of Stake	5
2.4.3 Proof of History	5
2.4.4 Byzantine Fault Tolerance	5
2.4.5 Classical Example of Byzantine Faults	5
2.5 Zero-Knowledge	6
2.6 Zero Knowledge with Graph Isomorphism	7
2.6.1 How does it work?	7
2.6.2 Toy Example	8
2.6.2.1 Step 1 — Prover	9
2.6.2.2 Case 1 - Verifier chooses $b\text{-prime} = 0$	9
2.6.2.3 Case 2 - Verifier chooses $b\text{-prime} = 1$	9

2.7	Zero-Knowledge Commitment Schemes	10
2.7.1	ElGamal Encryption Scheme	10
2.7.2	Disjunctive Chaum–Pedersen (Sigma-OR Proof)	12
2.7.3	The Chaum-Pedersen Interactive Protocol	12
2.7.3.1	Chaum–Pedersen Interactive Zero-Knowledge Protocol: Toy Example	14
2.7.4	The Chaum–Pedersen NIZK	17
2.7.4.1	Chaum–Pedersen Non-Interactive zero Knowledge Protocol : ToyExample	18
2.8	Zero-Knowledge Rollups	21
2.8.1	What does Zero-Knowledge mean in this context?	21
2.8.2	Efficiency	22
2.9	Conclusion	22
3	Technologies and Tools Used	23
3.1	Rust	23
3.2	Anchor	23
3.3	Tokio	23
3.4	Curve25519	24
3.5	Dalek Cryptography Libraries	24
3.5.1	Solana ZK SDK	25
3.6	Leptos	25
3.7	Github	25
3.8	Zed	25
3.9	Conclusion	26
4	Kaonashi from Scratch	27
4.1	Introduction	27
4.2	Libraries and Development Environment	29
4.3	Manual ElGamal Implementation	29
4.4	Vote Representation	30
4.5	Homomorphic Tally	31
4.6	Zero-Knowledge Proofs	31
4.6.1	VoteProof	32
4.6.2	VoteSumProof	32
4.6.3	Fiat-Shamir Transform	33
4.7	Smart Contract Design	33
4.7.1	Ballot Account	33
4.7.2	VoterRecord Account	34
4.7.3	Program Instructions	34
4.8	Testing the Prototype	35

4.8.1	Final Tally Decryption	36
4.8.2	Testing Kaonashi from Scratch	36
4.9	Conclusion	38
5	Kaonashi Scales Up	39
5.1	Introduction	39
5.1.1	Voting Flow	40
5.1.2	Toy Example for the Voting Flow	42
5.1.2.1	VoteProof	43
5.1.2.2	VoteSumProof	44
5.1.2.3	Why Both Proofs Are Needed	46
5.1.2.4	Backend Verification	46
5.1.3	Rollup	47
5.1.3.1	Off-chain Rollup Coordination	47
5.1.3.2	On-chain Rollup State	49
5.1.4	How do the Receipts Work?	52
5.1.5	Chairperson and its Power over the Election	52
5.2	Watching Kaonashi on Solana Explorer	55
5.2.1	Testing Kaonashi Scales Up	57
5.3	Frontend Interface	60
5.4	Conclusion	60
6	Conclusions and Future Work	61
6.1	Main Conclusions	61
6.2	Future Work	61
	Bibliography	63

List of Figures

2.1	Example of an isomorphism f between two graphs with 6 vertices.	7
4.1	Structure of the first Kaonashi prototype.	28
4.2	Validation flow inside the <code>cast_vote</code> instruction, showing the sequential checks that must all pass before a vote is folded into the encrypted tally.	35
4.3	Ballot initialization and voter registration.	37
4.4	Valid votes and blocked invalid voting attempts.	37
4.5	Final voter records and final winner.	38
5.1	Encrypted voting flow, before the rollup.	41
5.2	Merkle tree for a batch with four encrypted vote submissions. . . .	48
5.3	Compact rollup batch summary submitted from the API to the on-chain ballot.	49
5.4	Rollup flow: batches are formed and committed off-chain, then validated and folded into the ballot state on-chain.	51
5.5	Chairperson authorization flow in Kaonashi.	54
5.6	Transactions on Solana	55
5.7	Rollup batch submission for decade 0, containing the new Merkle root, encrypted batch tally, and batch size.	56
5.8	Final winner being stored on-chain after the encrypted tally was decrypted and finalized.	56
5.9	Chairperson creates the decade ballots.	57
5.10	Submission of ten encrypted votes.	57
5.11	Election closing, batch flushing, and tie detection.	58
5.12	Tie resolution by the chairperson.	58
5.13	Final result after tie resolution.	58
5.14	API logs showing vote proof verification and rollup batch submission.	59
5.15	API logs showing tie detection and finalization.	59
5.16	*	60
5.17	*	60
5.18	*	60
5.19	Main frontend pages of the improved Kaonashi system.	60

Code Snippets

Acronyms

BFT	Byzantine Fault Tolerance
CLI	Command-Line Interface
CS	Computer Science
ECC	Elliptic Curve Cryptography
ECDLP	Elliptic Curve Discrete Logarithm Problem
DDH	Decisional Diffie-Hellman
GI	Graph Isomorphism
IDL	Interface Definition Language
JSON	JavaScript Object Notation
NP	Nondeterministic Polynomial Time
P	Prover
V	Verifier
PDA	Program Derived Address
CPI	Cross-Program Invocation
BPF	Berkeley Packet Filter
PoH	Proof of History
PoS	Proof of Stake
PoW	Proof of Work
RPC	Remote Procedure Call
SDK	Software Development Kit
WASM	WebAssembly

TS	TypeScript
UBI	Universidade da Beira Interior
ZK	Zero-Knowledge
ZKP	Zero-Knowledge Proof
NIZK	Non-Interactive Zero-Knowledge
CP	Chaum–Pedersen Protocol
OR	Disjunctive (Sigma-OR) Proof
ElGamal	ElGamal Encryption Scheme
ZKR	Zero-Knowledge Rollup
RC	Rollup Coordinator
OHV	One-Hot vector

Glossary

One-hot vector A vector made entirely of 0s, except for a single 1 in one specific position. In Kaonashi, it is used to represent the selected proposal in a vote. For example, if a ballot has four proposals, a vote for the third proposal is represented as:

[0, 0, 1, 0]

Chapter

1

Introduction

1.1 Context

This technical report was developed within the scope of the Project course of the Bachelor's degree in Informatics Engineering at Universidade da Beira Interior (UBI).

The project focuses on Kaonashi, a privacy-preserving electronic voting system built on Solana. Its goal is to allow users to vote anonymously while keeping the election process verifiable, implementing a scalable layer-2 blockchain solution.

1.2 Motivation

Kaonashi explores how homomorphic encryption and zero-knowledge proofs can protect each vote while still allowing the system to verify and count it correctly. The project also studies how *rollups* can make this process scalable.

1.3 Objectives

The main objective of this project is to build Kaonashi, a private and verifiable voting system on Solana. To achieve this, the project aims to encrypt votes using ElGamal, validate encrypted ballots with zero-knowledge proofs, manage elections through an Anchor smart contract, use Solana's native cryptographic libraries, reduce on-chain costs through rollup-based vote aggregation, and test and visualize the voting process using Solana Explorer.

1.3.1 Why Kaonashi?

The name *Kaonashi* comes from the Japanese expression often associated with the idea of “No-Face” or the absence of an identifiable face. This name was chosen because it reflects the main goal of the system: allowing users to participate in a vote without exposing their individual identity or revealing their selected option.

1.4 Challenges and Contributions

The main challenge of this project lies in combining several complex areas: the theoretical study of zero-knowledge systems, asynchronous programming in Rust, and the practical development of blockchain-based applications.

The main contributions of this project are the following:

- A study of zero-knowledge systems, supported by small toy examples to better understand the underlying concepts.
- The development of a backend that connects cryptographic operations with blockchain interactions.
- The implementation of a frontend that allows users to interact with the voting system in a simple and accessible way.
- The integration of a rollup-based approach to improve scalability and reduce the amount of work performed directly on-chain.

1.5 Report Outline

The report is structured into six chapters:

1. **Introduction:** the projects motivation, challenges, and contributions.
2. **State of the Art:** introduces the main concepts related to e-voting, blockchain, zero-knowledge proofs, and rollups.
3. **Technologies and Tools Used:** presents the tools and technologies used.
4. **Kaonashi from Scratch:** describes the first version of the system.
5. **Kaonashi Scales Up:** presents the improved and more scalable version of Kaonashi.
6. **Conclusions and Future Work:** summarizes the project and possible future improvements.

Chapter

2

State of the Art

2.1 Introduction

In this chapter, the fundamental concepts and technologies underlying this project are presented. The focus is on blockchain systems, zero-knowledge proofs, and Layer 2 scalability solutions, particularly zkRollups, which enable scalable and privacy-preserving decentralized applications.

This chapter provides a brief overview of the key concepts required to understand the design of a zkRollup-based protocol for a privacy-preserving voting application, as well as the main techniques and tools used in modern zero-knowledge and blockchain systems.

2.2 Blockchain

A blockchain [1, 2] is a distributed and decentralized data structure that records information in a secure and immutable way. Instead of being stored on a central server, the ledger is replicated across multiple nodes in a network.

Data is organized into blocks, each containing a set of transactions, a cryptographic hash, and the hash of the previous block. This links the blocks in a chain and ensures that any modification to past data breaks the structure and is rejected by the network.

Nodes validate transactions and agree on the state of the blockchain through consensus mechanisms such as Proof of Work (PoW) or Proof of Stake (PoS). This process enables trust without relying on a central authority. [3]

2.3 Bitcoin

Bitcoin is particularly important because it was the first system to successfully achieve decentralized consensus without relying on a central authority. By solving the double spending problem using blockchain and Proof-of-Work, it opened the way for the development of modern blockchain systems and many subsequent innovations in distributed and cryptographic protocols.

In practice, transactions are grouped into blocks that are validated by miners through a Proof-of-Work mechanism. These blocks are linked together using cryptographic hashes, forming a blockchain that is hard to modify. Consensus is achieved by following the chain with the highest accumulated computational work, allowing the network to agree on a single version of the ledger even in the presence of malicious participants. [3]

2.4 Solana

Solana [1] is a blockchain that uses a Proof of History (PoH) consensus mechanism.

Data transformation in Solana is performed using the SHA-256 hash function which generates a unique combination of characters from the input data. The network creates the next hash of data based on the received transaction information. The process described above creates a continuous, long chain of hashed transactions, which is relatively easy to understand and verify. This hashing algorithm is what makes transaction processing secure and reliable.

Ethereum is currently the most prominent blockchain. It hosts more decentralized applications than any other blockchain ecosystem, however Ethereum only has a capacity of 15 transactions per second whilst Solana provides one of the highest transaction processing speed per second.

The Solana blockchain combines several protocols aiming to address the blockchain trilemma of scalability, security and decentralization. In Solana it only takes about 0.4 seconds to process a block, and as hardware capacity increases, the speed of the entire network can also improve. The difference in speed between Ethereum and Solana is largely due to the difference in the consensus algorithms used.

2.4.1 Proof of Work

Proof of Work (PoW) consensus blockchains require a large amount of energy-consuming computational work. Due to current environmental concerns, many consider the PoW consensus mechanism inefficient in terms of energy

consumption. Low energy efficiency was one of the main reasons Ethereum transitioned from Proof of Work (PoW) to Proof of Stake (PoS). [4]

2.4.2 Proof of Stake

Proof of Stake (PoS) is a consensus mechanism in which validators are selected to create new blocks based on the amount of cryptocurrency they stake. Unlike Proof of Work, it does not require solving computational puzzles, making it significantly more energy efficient.

Validators are chosen randomly, and the more they stake, the higher their probability of being selected to propose a block. If selected, they are rewarded for proposing valid blocks. Malicious behavior can be penalized through mechanisms such as slashing, which helps maintain network security. [4]

2.4.3 Proof of History

Solana introduces the concept of Proof of History (PoH), which is not a consensus mechanism but rather a cryptographic technique used to establish time within the blockchain.

PoH creates a sequential chain of hashes, where each hash depends on the previous one, forming a verifiable timeline. Since each hash can only be computed after the previous one, it becomes possible to measure the passage of time between events.

When a transaction is submitted to the network, it is inserted into a specific position within this sequence. This allows the system to determine the exact order of transactions without requiring all nodes to communicate and agree on ordering. [4]

2.4.4 Byzantine Fault Tolerance

Solana also uses a Byzantine Fault Tolerance (BFT) mechanism to address a classical problem in distributed systems, where some nodes may behave maliciously or fail.

In such systems, the network can continue to operate correctly as long as no more than one-third of the nodes are faulty or malicious. This ensures reliability and security even in the presence of adversarial participants. [5]

2.4.5 Classical Example of Byzantine Faults

The Byzantine Generals Problem, introduced by Lamport, Shostak, and Pease, illustrates the difficulty of achieving consensus in a distributed system when

some participants may fail or behave maliciously. In the classical example, several generals surround a city and must agree on a common action, such as attack or retreat. Each general communicates with the others through messengers, but some of the generals may be traitors, meaning that they can send different or misleading information to different participants.

The goal is to guarantee three main properties. First, agreement, which means that all loyal generals must reach the same decision. Second, validity, which means that if the commander is loyal, then all loyal generals must follow the order sent by the commander. Third, termination, which means that all loyal generals must eventually decide on some action.

For example, suppose that the commander sends the order retreat to all generals. If one of the generals is a traitor, he may forward a different message, such as attack, to some of the others in order to create confusion. As a result, loyal generals may receive contradictory information: the original order retreat from the commander and a false order attack from the traitor. The challenge is therefore to design a protocol that allows all loyal generals to still reach the same final decision despite the presence of conflicting messages. [6]

2.5 Zero-Knowledge

Zero-knowledge proofs (ZKPs) [7] are cryptographic techniques that allow a statement to be verified without revealing the underlying information. Basically a prover demonstrates the validity of a claim to a verifier, who can confirm its correctness without gaining any additional knowledge.

ZKPs can be classified as interactive or non-interactive. Interactive protocols require multiple rounds of communication between the prover and the verifier to increase confidence in the result, whereas non-interactive proofs allow verification in a single step.

ZKPs provide probabilistic assurance rather than absolute certainty. By repeating the protocol multiple times, the probability of accepting a false statement becomes negligible [7].

For a proof system to be considered zero-knowledge, it must satisfy three fundamental properties [8] :

- **Completeness:** If the statement is true, the verifier will accept the proof.
- **Soundness:** If the statement is false, the verifier will reject the proof, except with negligible probability.
- **Zero-knowledge:** The verifier learns nothing beyond the validity of the statement.

In the next section an example of a non-trivial Zero Knowledge system is given.

2.6 Zero Knowledge with Graph Isomorphism

Two graphs G_0 and G_1 are isomorphic if there exists a permutation of their vertices that transforms one graph into the other while preserving the edges.

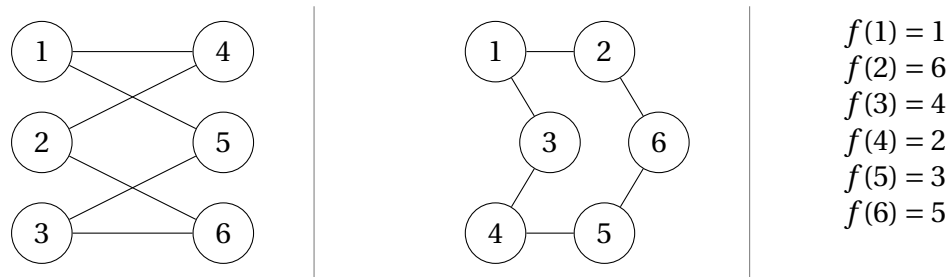


Figure 2.1: Example of an isomorphism f between two graphs with 6 vertices.

Our computational problem is the Graph Isomorphism problem, which consists of determining whether two finite graphs are isomorphic. It belongs to the Nondeterministic Polynomial Time (NP) class and currently has a quasi-polynomial time algorithm. [9]

In our protocol, the Prover (P) wants to prove that he knows the permutation π such that $\pi(G_0) = G_1$, where the permutation π simply renames the vertices. However, P does not want to reveal π . Instead, he wants to prove that he knows the solution without revealing it, as required in a **Zero-Knowledge proof** [9].

To accomplish this, we repeatedly shuffle the graph so that the verifier only sees shuffled versions of the graphs and never learns the real permutation.

2.6.1 How does it work?

The prover creates a shuffled graph H by randomly choosing a permutation σ and a bit $b \in \{0, 1\}$. He then selects one of the two graphs, G_0 or G_1 , according to the value of b , and computes:

$$H = \sigma(G_b)$$

In this way, the verifier only sees H and does not know whether it was obtained from G_0 or G_1 .

The verifier then chooses a challenge bit $b' \in \{0, 1\}$ and asks the prover to show that H is isomorphic to $G_{b'}$.

The prover then answers with a permutation τ that depends on the values of b and b' :

$$\tau = \begin{cases} \sigma & \text{if } b = b' \\ \sigma\pi^{-1} & \text{if } b = 0, b' = 1 \\ \sigma\pi & \text{if } b = 1, b' = 0 \end{cases}$$

The verifier then checks whether:

$$H = \tau(G_{b'})$$

If this equality holds, the verifier accepts the proof for that round; otherwise, the verifier rejects.

This protocol works because if the prover truly knows the isomorphism π such that $\pi(G_0) = G_1$, he can always construct the correct permutation τ to satisfy the verifier's challenge. However, the verifier learns **nothing** (Zero!) about the actual isomorphism π , since the graph H and the permutation τ reveal only that an isomorphism exists, not what it is.

2.6.2 Toy Example

Consider two isomorphic graphs G_0 and G_1 with vertices $\{1, 2, 3, 4, 5, 6\}$ as shown in the example above.

The secret isomorphism is:

$$\pi : 1 \rightarrow 1, 2 \rightarrow 6, 3 \rightarrow 4, 4 \rightarrow 2, 5 \rightarrow 3, 6 \rightarrow 5$$

This corresponds to the permutation matrix

$$P = \begin{pmatrix} 1 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 1 \\ 0 & 0 & 0 & 1 & 0 & 0 \\ 0 & 1 & 0 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 1 & 0 \end{pmatrix}$$

A permutation matrix is a square binary matrix that has exactly one 1 in each row and column, with zeros elsewhere, used to reorder rows or columns. They have interesting properties such as the inverse of a permutation matrix is its

transpose, the determinant is either 1 or -1 and they also orthogonal matrices. The product of two permutation matrices is itself a permutation matrix, simply the composition of two permutations. In the following example we can represent the graphs as adjacency matrices and compute the operations doing matrix operations - however we omit these calculations here.

2.6.2.1 Step 1 — Prover

The prover chooses:

- $b = 0$
- a random permutation $\sigma = (1 \rightarrow 2, 2 \rightarrow 3, 3 \rightarrow 1, 4 \rightarrow 5, 5 \rightarrow 6, 6 \rightarrow 4)$

Then computes $H = \sigma(G_0)$ and sends H to the verifier.

2.6.2.2 Case 1 - Verifier chooses $b\text{-prime} = 0$

Since $b = b'$, the prover sends:

$$\tau = \sigma$$

The verifier checks:

$$H = \tau(G_0)$$

2.6.2.3 Case 2 - Verifier chooses $b\text{-prime} = 1$

Now $b \neq b'$, so the prover sends:

$$\tau = \sigma\pi^{-1}$$

The verifier checks:

$$H = \tau(G_1) = \sigma\pi^{-1}(G_1) = \sigma(G_0)$$

The prover can always answer correctly only if he knows π , but the verifier never learns what π actually is. In this case a malicious verifier only has a slightly large than 50 chance of correctly convincing the verifier he knows the secret. To be more precise it's $\frac{1}{2} + \frac{1}{6} = \frac{2}{3}$.

This protocol illustrates the three main properties of zero-knowledge proofs. Completeness means that if the prover knows the isomorphism π , he can always convince the verifier. Soundness means that if the prover does not know π , he can only guess correctly with probability at most $\frac{1}{2}$ in each round. Finally, zero-knowledge means that the verifier learns nothing about π , only that such a permutation exists.

2.7 Zero-Knowledge Commitment Schemes

2.7.1 ElGamal Encryption Scheme

The ElGamal encryption scheme [10] is a public-key cryptosystem whose security is based on the Decisional Diffie-Hellman (DDH) assumption in a cyclic group G of order q , with generator g .

The key generation process is defined as follows. A secret key $x \in \mathbb{Z}_q$ is chosen uniformly at random, and the corresponding public key is computed as:

$$h = g^x$$

Thus, the public key is given by (G, q, g, h) , while the private key is x .

To encrypt a message $m \in G$, the sender samples a fresh random value $y \in \mathbb{Z}_q$ and computes the ciphertext:

$$(c_1, c_2) = (g^y, h^y \cdot m)$$

Decryption is performed using the private key x as:

$$m = \frac{c_2}{c_1^x}$$

Since $c_1^x = g^{xy}$, we obtain:

$$\frac{h^y \cdot m}{g^{xy}} = m$$

This scheme ensures semantic security under the DDH assumption, meaning that ciphertexts do not reveal any information about the underlying message.

A crucial property of ElGamal encryption is its multiplicative homomorphism. Given two ciphertexts:

$$(c_1, c_2) = (g^{y_1}, h^{y_1} \cdot m_1), \quad (d_1, d_2) = (g^{y_2}, h^{y_2} \cdot m_2)$$

their component-wise multiplication yields:

$$(c_1 d_1, c_2 d_2) = (g^{y_1+y_2}, h^{y_1+y_2} \cdot m_1 m_2)$$

which corresponds to an encryption of $m_1 \cdot m_2$. In voting systems, this property is typically adapted into an additive form via appropriate encoding.

Toy Example in ElGamal Voting

Consider a vote encrypted as:

$$(c_1, c_2) = (g^y, h^y \cdot g^m)$$

where $m \in \{0, 1\}$.

To prove validity, the prover must show:

$$\log_g(c_1) = y \quad \text{AND} \quad \log_h\left(\frac{c_2}{g^m}\right) = y$$

but without revealing whether $m = 0$ or $m = 1$.

The idea of the Sigma-OR construction is to combine two Chaum–Pedersen proofs:

- a **real proof** for the actual value of m ,
- a **simulated proof** for the other value.

The prover generates commitments for both cases:

- for the real statement, the proof is computed honestly;
- for the false statement, the proof is simulated by choosing the challenge in advance and constructing a valid transcript.

The global challenge is then split as:

$$c = c_0 + c_1$$

and responses are computed accordingly:

$$s_i = r_i + c_i \cdot w_i$$

where w_i is the witness for the real statement.

The verifier checks both Chaum–Pedersen relations:

$$g^{s_i} = t_i \cdot y_i^{c_i}$$

for $i \in \{0, 1\}$, and accepts only if both hold and the challenge is consistent.

Conclusion: The verifier becomes convinced that the ciphertext encrypts either 0 or 1, while learning nothing about which value was chosen.

2.7.2 Disjunctive Chaum–Pedersen (Sigma-OR Proof)

In binary voting systems, it is not sufficient to only encrypt a value; we must also ensure that the encrypted message is valid, i.e., it is either 0 or 1, without revealing which one was chosen.

Let a voter submit an ElGamal ciphertext (c_1, c_2) . The goal is to prove the following statement:

$$m \in \{0, 1\}$$

without revealing m .

This is achieved using the Disjunctive Chaum–Pedersen protocol, also known as the Sigma-OR proof, which allows proving that at least one of two discrete-log statements is true.

2.7.3 The Chaum–Pedersen Interactive Protocol

Proposed by David Chaum and Torben Pedersen [11], the Chaum–Pedersen protocol is a zero-knowledge proof system that allows a Prover to demonstrate knowledge of a secret value without revealing the secret itself.

In the context of electronic voting systems, this protocol enables voters to prove that a vote has been correctly formed while preserving vote privacy. More formally, the protocol allows a Prover to prove equality between two discrete logarithms without revealing the underlying secret value. This type of proof is commonly referred to as a *Discrete Log Equality* (DLEQ) proof and belongs to the family of Σ -protocols.

The interactive protocol proceeds as follows.

Step 1: Protocol Setup

Let G be a cyclic group of prime order q with generators g and h .

Goal

- The Prover wishes to prove to the Verifier that they know a secret value x .
- The Verifier knows the public values:

$$y = g^x \quad \text{and} \quad z = h^x$$

The Prover therefore wants to prove that:

$$\log_g y = \log_h z = x$$

without revealing the secret value x .

Step 2: Commitment

The Prover selects a random nonce $k \in \mathbb{Z}_q$ and computes the commitment pair (a, b) :

$$a = g^k \tag{2.1}$$

$$b = h^k \tag{2.2}$$

The Prover then sends (a, b) to the Verifier.

Step 3: Challenge

The Verifier selects a random challenge $c \in \mathbb{Z}_q$ and sends it to the Prover:

$$c \xleftarrow{R} \mathbb{Z}_q \tag{2.3}$$

Step 4: Response

Using the secret value x , the nonce k , and the challenge c , the Prover computes:

$$s = k + cx \pmod{q} \tag{2.4}$$

The Prover then sends s to the Verifier.

Step 5: Verification

The Verifier accepts the proof if and only if the following equalities hold:

$$g^s \equiv a \cdot y^c \pmod{p} \tag{2.5}$$

$$h^s \equiv b \cdot z^c \pmod{p} \tag{2.6}$$

Verification Logic

To verify correctness (completeness), observe that:

$$a \cdot y^c = g^k \cdot (g^x)^c = g^{k+cx} = g^s$$

$$b \cdot z^c = h^k \cdot (h^x)^c = h^{k+cx} = h^s$$

Therefore, an honest Prover will always satisfy the verification equations.

Soundness: A dishonest Prover who does not know the secret value x cannot correctly answer multiple random challenges for the same commitment pair (a, b) . Consequently, forging a valid proof becomes computationally infeasible.

Zero-Knowledge: The protocol reveals no information about the secret value x . A simulator can generate a transcript (a, b, c, s) that is computationally indistinguishable from a real protocol execution without knowing x , demonstrating the zero-knowledge property.

Appendix 2.7.3.1 gives a fully worked **Toy example** over a small prime field.

2.7.3.1 Chaum–Pedersen Interactive Zero-Knowledge Protocol: Toy Example

In this example, the Prover wants to interactively prove that they know a secret value x such that:

$$y = g^x \quad \text{and} \quad z = h^x$$

without revealing the value of x .

Step 1: Setup For simplicity, let us consider the following small parameters:

$$p = 23, \quad g = 2, \quad h = 3$$

Assume that the Prover's secret value is:

$$x = 4$$

The public values are computed as follows:

$$y = g^x = 2^4 = 16 \pmod{23}$$

Thus:

$$y = 16$$

Similarly,

$$z = h^x = 3^4 = 81 \equiv 12 \pmod{23}$$

Thus:

$$z = 12$$

At this stage, the public statement is:

$$(g, h, y, z) = (2, 3, 16, 12)$$

while the secret value $x = 4$ remains known only to the Prover.

Step 2: Commitment The Prover chooses a random nonce:

$$k = 5$$

and computes the commitment values:

$$a = g^k = 2^5 = 32 \equiv 9 \pmod{23}$$

Thus:

$$a = 9$$

Similarly,

$$b = h^k = 3^5 = 243 \equiv 13 \pmod{23}$$

Thus:

$$b = 13$$

The Prover sends $(a, b) = (9, 13)$ to the Verifier.

Step 3: Challenge Unlike the non-interactive version, the challenge is not computed by hashing the public parameters. Instead, the Verifier picks a random challenge and sends it directly to the Prover.

For this toy example, let us assume that the Verifier chooses:

$$c = 2$$

and sends this value back to the Prover.

Step 4: Response The Prover computes:

$$s = k + cx$$

Substituting the values:

$$s = 5 + 2 \cdot 4 = 13$$

Thus:

$$s = 13$$

The Prover sends $s = 13$ to the Verifier.

Step 5: Verification The Verifier already knows the challenge $c = 2$, since it was chosen by the Verifier in Step 3. The Verifier now checks the first equation:

$$g^s \equiv a \cdot y^c \pmod{23}$$

Left-hand side:

$$g^s = 2^{13} = 8192 \equiv 4 \pmod{23}$$

Right-hand side:

$$a \cdot y^c = 9 \cdot 16^2$$

Since:

$$16^2 = 256 \equiv 3 \pmod{23}$$

we obtain:

$$9 \cdot 3 = 27 \equiv 4 \pmod{23}$$

Thus:

$$g^s \equiv a \cdot y^c \pmod{23}$$

The Verifier now checks the second equation:

$$h^s \equiv b \cdot z^c \pmod{23}$$

Left-hand side:

$$h^s = 3^{13} \equiv 9 \pmod{23}$$

Right-hand side:

$$b \cdot z^c = 13 \cdot 12^2$$

Since:

$$12^2 = 144 \equiv 6 \pmod{23}$$

we obtain:

$$13 \cdot 6 = 78 \equiv 9 \pmod{23}$$

Thus:

$$h^s \equiv b \cdot z^c \pmod{23}$$

Since both equations hold, the Verifier accepts the proof and is convinced that the Prover knows the secret value x , without learning anything about it.

Unlike the non-interactive version, this exchange required the Verifier to actively participate by choosing and sending the challenge c before the Prover could compute the response s .

2.7.4 The Chaum–Pedersen NIZK

The Chaum–Pedersen protocol can be transformed into a non-interactive zero-knowledge proof (NIZK) using the Fiat–Shamir heuristic [12]. A detailed discussion of this transformation can also be found in Boneh and Shoup’s *Graduate Course in Applied Cryptography* [8].

In the interactive version of the protocol, the Verifier generates the random challenge c . The Fiat–Shamir heuristic removes this interaction by deriving the challenge deterministically from a cryptographic hash of the public statement and the commitment values.

Instead of receiving c from the Verifier, the Prover computes:

$$c = H(g, h, y, z, a, b) \tag{2.7}$$

where H is a secure cryptographic hash function such as SHA-256.

By hashing all public parameters and commitment values, the challenge becomes bound to the specific proof instance. This prevents replay attacks and prevents the proof from being reused for different values or statements.

The resulting proof is therefore non-interactive, since any verifier can independently recompute the challenge and validate the proof without communicating with the Prover.

2.7.4.1 Chaum–Pedersen Non-Interactive zero Knowledge Protocol : ToyExample

In this example, the Prover wants to prove non-interactively that they know a secret value x such that:

$$y = g^x \quad \text{and} \quad z = h^x$$

without revealing the value of x .

Step 1: Setup For simplicity, let us consider the following small parameters:

$$p = 23, \quad g = 2, \quad h = 3$$

Assume that the Prover's secret value is:

$$x = 4$$

The public values are computed as follows:

$$y = g^x = 2^4 = 16 \pmod{23}$$

Thus:

$$y = 16$$

Similarly,

$$z = h^x = 3^4 = 81 \equiv 12 \pmod{23}$$

Thus:

$$z = 12$$

At this stage, the public statement is:

$$(g, h, y, z) = (2, 3, 16, 12)$$

while the secret value $x = 4$ remains known only to the Prover.

Step 2: Commitment The Prover chooses a random nonce:

$$k = 5$$

and computes the commitment values:

$$a = g^k = 2^5 = 32 \equiv 9 \pmod{23}$$

Thus:

$$a = 9$$

Similarly,

$$b = h^k = 3^5 = 243 \equiv 13 \pmod{23}$$

Thus:

$$b = 13$$

Step 3: Non-Interactive Challenge Using the Fiat–Shamir heuristic, the challenge is generated non-interactively by hashing the public parameters and the commitment values:

$$c = H(g, h, y, z, a, b)$$

For this toy example, let us assume that:

$$H(2, 3, 16, 12, 9, 13) = 2$$

In practice, this value would be computed using a secure cryptographic hash function such as SHA-256.

Step 4: Response The Prover computes:

$$s = k + cx$$

Substituting the values:

$$s = 5 + 2 \cdot 4 = 13$$

Thus:

$$s = 13$$

The final proof is therefore:

$$(a, b, s) = (9, 13, 13)$$

Step 5: Verification The Verifier recomputes the challenge:

$$c = H(g, h, y, z, a, b)$$

obtaining:

$$c = 2$$

The Verifier then checks the first equation:

$$g^s \equiv a \cdot y^c \pmod{23}$$

Left-hand side:

$$g^s = 2^{13} = 8192 \equiv 4 \pmod{23}$$

Right-hand side:

$$a \cdot y^c = 9 \cdot 16^2$$

Since:

$$16^2 = 256 \equiv 3 \pmod{23}$$

we obtain:

$$9 \cdot 3 = 27 \equiv 4 \pmod{23}$$

Thus:

$$g^s \equiv a \cdot y^c \pmod{23}$$

The Verifier now checks the second equation:

$$h^s \equiv b \cdot z^c \pmod{23}$$

Left-hand side:

$$h^s = 3^{13} \equiv 9 \pmod{23}$$

Right-hand side:

$$b \cdot z^c = 13 \cdot 12^2$$

Since:

$$12^2 = 144 \equiv 6 \pmod{23}$$

we obtain:

$$13 \cdot 6 = 78 \equiv 9 \pmod{23}$$

Thus:

$$h^s \equiv b \cdot z^c \pmod{23}$$

Since both equations hold, the Verifier accepts the proof and is convinced that the Prover knows the secret value x , without learning anything about it.

2.8 Zero-Knowledge Rollups

A Rollup is a Layer 2 scaling solution that processes transactions off-chain and submits a compressed result to the main blockchain, reducing transaction fees and network congestion [13].

In this model, transactions are executed off-chain in batches by a Rollup Coordinator (RC). There are two types of model, zero-knowledge rollups (zk-Rollups) and SNARK based optimistic RollUps.

Instead of sending all transaction data to the main chain, only a succinct representation of the updated state, together with a cryptographic proof, is submitted. This significantly reduces the amount of data stored and processed on-chain.

zkRollups rely on zero-knowledge [14] proofs to guarantee that all off-chain computations are correct. Rather than trusting the operator, the system relies on cryptographic verification, ensuring that the state transitions are valid.

2.8.1 What does Zero-Knowledge mean in this context?

In this setting, zero-knowledge [14] refers to the generation of a cryptographic proof that demonstrates that a batch of transactions was executed correctly, without revealing any sensitive information about those transactions.

For example, Ethereum can verify the validity of an entire batch by checking a single proof, instead of re-executing each transaction individually. This provides correctness guarantees while preserving privacy.

2.8.2 Efficiency

zkRollups are highly efficient because they avoid re-executing transactions on-chain. By moving computation off-chain and only verifying a proof on-chain, they significantly reduce computational overhead.

Additionally, zkRollups provide faster transaction finality compared to optimistic rollups,[15] since transactions are considered valid as soon as the proof is verified. There is no need to wait for fraud detection periods.

This approach allows the network to scale efficiently while maintaining strong security guarantees, as every state update must be validated through cryptographic proofs.[13]

2.9 Conclusion

This chapter presented the key concepts behind Kaonashi, including blockchain, consensus mechanisms, zero-knowledge proofs, and zkRollups. These concepts show how privacy, verifiability, and scalability can be combined in a voting system, forming the foundation for the work developed in Kaonashi.

Chapter

3

Technologies and Tools Used

The tools and technologies used in this project are presented in this chapter.

3.1 Rust

Solana is primarily developed using Rust, a language chosen for its high performance, memory safety, and support for secure concurrency without relying on a garbage collector. Therefore, using Rust helps maintain consistency with the Solana ecosystem. [16]

3.2 Anchor

Anchor is a framework built on top of Rust that enables the development of smart contracts on Solana. It automatically validates accounts and instruction data, reducing common vulnerabilities. Additionally, it reduces boilerplate code through the use of macros. Anchor crucially provides a command-line interface (CLI) and a local testing environments. [17]

3.3 Tokio

Tokio is as asynchronous runtime for Rust. In this project, Tokio is essential because we interact with the blockchain using asynchronous operations, such as sending transactions and reading account data. Tokio acts as the runtime that enables this asynchronous execution using Rust's `async/await` model. [18]

3.4 Curve25519

An elliptic curve is a set of mathematical points that satisfy a specific equation and form an algebraic structure that can be used in cryptographic protocols. Curve25519 is an elliptic curve designed to provide high cryptographic security, computational efficiency, resistance against known attacks, and simple yet secure implementations. It was originally designed for use with the Elliptic Curve Diffie-Hellman (ECDH) key agreement scheme.

The original form of the curve is given by:

$$y^2 = x^3 + 486662x^2 + x$$

where arithmetic operations are performed over the prime field:

$$p = 2^{255} - 19$$

This prime number is the reason for the name *Curve25519*. The number 255 refers to the size of the prime field, while 19 appears in the definition of the prime $2^{255} - 19$.

In Kaonashi, Curve25519 is not implemented directly. The project does not manually compute points on the curve or implement the curve equation from scratch. Instead, Curve25519 is used indirectly through the cryptographic libraries of the Solana ecosystem, especially the Solana ZK SDK and Curve25519/Ristretto-based operations.

This is important in the *level-up version* of Kaonashi, where the manually implemented ElGamal scheme is replaced by library-based cryptographic primitives.

The Solana cryptographic ecosystem also relies on Ristretto, a prime-order group abstraction built from Edwards25519, which is closely related to Curve25519. Therefore, in Kaonashi, Curve25519 and Ristretto provide the mathematical foundation for encrypted votes, homomorphic tally aggregation, and zero-knowledge validation. The project relies on these primitives through existing Solana libraries, rather than implementing the curve operations manually.

3.5 Dalek Cryptography Libraries

Kaonashi integrates Dalek cryptographic libraries in two contexts. First, for **Curve25519 and Ristretto operations**, the curve25519-dalek library handles elements over Curve25519, Edwards25519, and the Ristretto group; Kaonashi accesses these primitives indirectly through the Solana ecosystem's

cryptographic tools rather than implementing them manually. Second, for **wallet authentication**, the project utilizes `ed25519-dalek` for digital signatures. During authentication, the user signs a challenge message with their private key, allowing the backend to verify ownership via the corresponding public key without transmitting the private key to the server.

3.5.1 Solana ZK SDK

The Solana ZK SDK serves as the primary cryptographic library for handling encrypted votes, providing ElGamal encryption over Ristretto-based curve operations. By representing each vote as a ciphertext, the system enables homomorphic aggregation, allowing encrypted votes to be summed without decrypting individual choices. The SDK manages ElGamal keypair generation, vote vector encryption, ciphertext format validation, encrypted tally aggregation, and the final decryption once voting closes. This architecture ensures individual vote privacy while maintaining a verifiable final tally.

3.6 Leptos

Leptos is a full-stack web framework for Rust that compiles to WebAssembly (WASM). It is used to implement the graphical frontend interface. In Kaonashi, Leptos handles user interactions such as submitting votes, communicating with the Solana RPC endpoint, with the axum based rollup coordinator, and displaying transaction results. The frontend is compiled to WebAssembly and executed in the browser. Trunk is name of the tool that is used to build and serve leptos applications.

3.7 Github

GitHub was used for version control and source code organization

3.8 Zed

Zed was used as the code editor during development, chosen for its speed, simple interface, and good support for Rust projects. This made it suitable for working across the backend, frontend, and smart contract code.

3.9 Conclusion

This chapter presented the main tools used to build Kaonashi. Rust, Anchor, and Tokio support the Solana smart contract and blockchain interaction, while the Solana ZK SDK provides the cryptographic tools needed for encrypted voting and tally aggregation. Leptos was used for the frontend, with GitHub and Zed supporting development. Together, these tools form the technical basis for the implementation.

Chapter

4

Kaonashi from Scratch

4.1 Introduction

This chapter documents the first implementation of Kaonashi, built with Solana and Anchor, where almost every cryptographic component was written by hand instead of relying on existing libraries: ElGamal encryption, homomorphic vote aggregation, and zero-knowledge proofs for vote validation.

The goal of this version was to understand, line by line, how encrypted voting can work on a blockchain. Doing everything manually is slower, but it helps to understand: how a vote gets encrypted, how encrypted votes can be summed without being decrypted, and how a smart contract can reject an invalid or dishonest vote without ever seeing what that vote actually says.

With that in mind, this first prototype had four concrete goals:

- encrypt votes before they ever touch the blockchain;
- aggregate those encrypted votes homomorphically;
- validate encrypted votes without revealing their plaintext content;
- block invalid votes, tampered ciphertexts, and double voting.

Voters were simulated through off-chain Rust scripts, their votes were encrypted before submission, the smart contract checked the attached proofs, and the encrypted tally lived on-chain until the very end of the election, when it was finally decrypted off-chain.

To keep things manageable, the first version of Kaonashi was split into a handful of clearly separated modules. The cryptographic module is where the ElGamal scheme actually lives: key generation, encryption, decryption,

vote encoding, and homomorphic aggregation. The proof module handles the verification logic behind the zero-knowledge vote proofs. The smart contract module defines everything that happens on-chain: the election state and the voting instructions. Finally, the testing module ties everything together off-chain, generating encrypted votes and proofs, firing transactions in a local Solana test environment, and decrypting the final tally once the election is over.

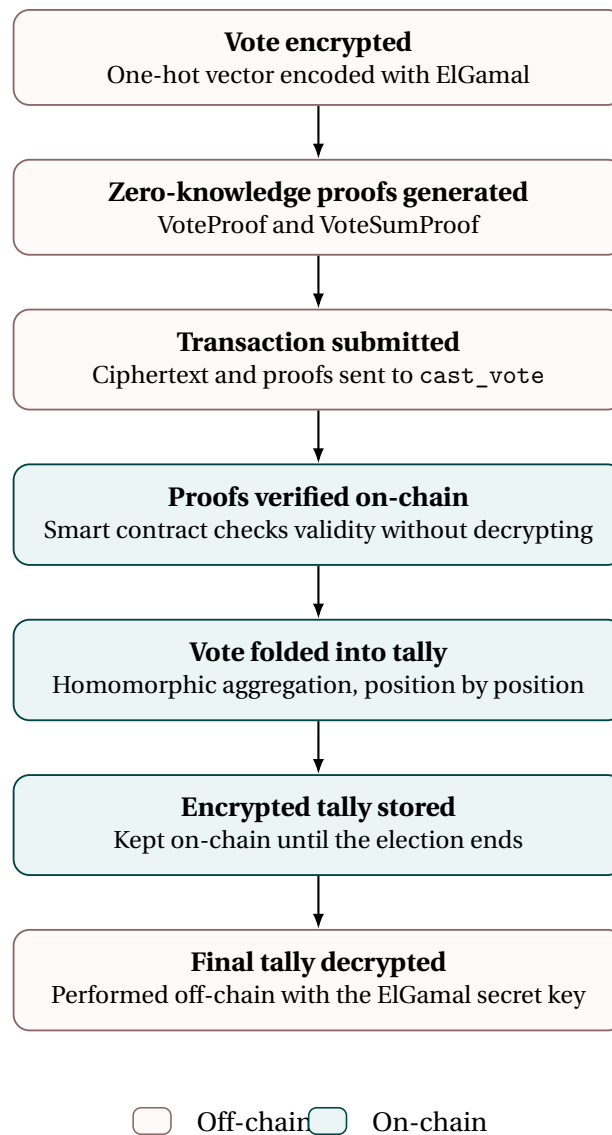


Figure 4.1: Structure of the first Kaonashi prototype.

4.2 Libraries and Development Environment

On the smart contract side, everything was built with Anchor, using the `anchor_lang` library to define program instructions, account contexts, account serialization, and custom errors. This gave the program a clean, instruction-based shape, with entry points like `initialize`, `register_voter`, `cast_vote`, and `set_final_winner`.

For hashing, the prototype used SHA-256 via Solana's own hashing library, hashing is exactly what makes the Fiat-Shamir transform work. Instead of waiting for a verifier to send a random challenge, the contract simply hashes the public data from the proof, the ciphertext, and the public key, and uses that hash as the challenge.

On the testing side, `solana_program_test` and `solana_sdk` made it possible to run the entire voting flow locally. A typical test run creates a ballot, registers voters, generates encrypted votes and proofs, submits the corresponding transactions, checks that bad votes get rejected, reads back the final on-chain state, and finally decrypts the tally off-chain.

4.3 Manual ElGamal Implementation

In this version is a hand-written implementation of the ElGamal, built around `PublicKey`, `SecretKey`, and `Ciphertext`.

The public key carries the group parameters p , q , g , together with the public value h . The secret key simply holds the private value x . The public value is derived from the private one as:

$$h = g^x \bmod p$$

A ciphertext is a pair of values, c_1 and c_2 . To encrypt a message m , the implementation draws a new random value r and computes:

$$c_1 = g^r \bmod p$$

$$c_2 = h^r \cdot m \bmod p$$

so that an encrypted message is, in the end, nothing more than the pair:

$$C = (c_1, c_2)$$

Decryption goes the other way: the secret key first recovers the shared mask,

$$s = c_1^x \bmod p$$

and the original message reappears once c_2 is multiplied by the modular inverse of that mask:

$$m = c_2 \cdot s^{-1} \bmod p$$

All of the supporting modular arithmetic, modular exponentiation, modular inverse, and so on, was implemented directly in Rust rather than borrowed from a library. This made the whole cryptographic flow fully transparent, which was exactly the point during this first phase of the project.

4.4 Vote Representation

Votes are represented as one-hot vectors: if a ballot offers three proposals, a vote for the second one simply looks like this:

$$[0, 1, 0]$$

Each slot in the vector corresponds to one proposal, and a valid vote has exactly one slot set to 1, with every other slot at 0.

Crucially, votes are not encrypted as raw integers. Instead, each value is first lifted into a group element:

$$m = g^v \bmod p$$

where v is either 0 or 1. In practice, this gives a very simple encoding:

$$0 \mapsto g^0 = 1$$

$$1 \mapsto g^1 = g$$

This small trick is what unlocks the homomorphic property of ElGamal later on. Each entry of the vote vector is encrypted separately, with its own randomness, producing something like:

$$[E(0), E(1), E(0)]$$

4.5 Homomorphic Tally

ElGamal happens to be multiplicatively homomorphic, this meaning that if you multiply two ciphertexts together, you get a new ciphertext whose plaintext is the product of the two original plaintexts. Given:

$$C = (c_1, c_2)$$

and

$$C' = (c'_1, c'_2)$$

their combination is simply:

$$C \cdot C' = (c_1 \cdot c'_1, c_2 \cdot c'_2)$$

This multiplication on ciphertexts turns into addition of vote counts in the exponent:

$$E(g^{v_1}) \cdot E(g^{v_2}) = E(g^{v_1+v_2})$$

This is the whole trick that lets the smart contract keep a running tally without ever decrypting a single vote. The encrypted tally is stored as a vector with one ciphertext per proposal, and every accepted vote simply gets folded in, position by position:

$$T_i \leftarrow T_i \cdot V_i$$

where T_i is the encrypted tally for proposal i , and V_i is the corresponding encrypted vote component. The tally starts out as a vector of encrypted zeroes,

$$[E(0), E(0), \dots, E(0)]$$

and grows as valid votes are accumulated into it, one ballot at a time.

4.6 Zero-Knowledge Proofs

The smart contract receives ciphertexts it cannot read, which is exactly the privacy property we want, but it also means a dishonest voter could try to sneak in an invalid value, or attempt to vote for more than one proposal at once, and the contract would have no way of noticing just by looking at the encrypted bytes.

The first version of Kaonashi implements two zero-knowledge proofs:

- `VoteProof`, which proves that a given encrypted component is either 0 or 1, nothing else;
- `VoteSumProof`, which proves that the entire encrypted vote vector adds up to exactly 1.

Put together, these two proofs are what guarantee that an encrypted vote really is a valid one-hot vector, without the contract ever learning which option was chosen.

4.6.1 `VoteProof`

`VoteProof` travels alongside each ciphertext in the vote vector, and its job is to convince the contract that the ciphertext encrypts either 0 or 1, without giving away which of the two it actually is. Conceptually, it proves a disjunction:

$$C = E(0) \quad \vee \quad C = E(1)$$

This is a classic "OR" proof: it has two branches, one assuming the ciphertext encrypts 0, the other assuming it encrypts 1. Only one of the two branches corresponds to the truth; the other is simulated so that, from the outside, both branches look equally convincing. The verifier checks both branches and confirms that their individual challenges are tied together through a shared global challenge.

4.6.2 `VoteSumProof`

`VoteSumProof` steps in to catch a problem that `VoteProof` alone does not solve: even if every single component of the vote vector is individually a valid 0 or 1, the vector as a whole can still be nonsense. Take, for example:

$$[1, 1, 0]$$

Every entry here is binary, yet the vote effectively selects two proposals at once. The opposite failure mode is just as easy to produce:

$$[0, 0, 0]$$

a vector that, despite being technically valid component-by-component, selects nothing at all.

To rule both cases out, the contract multiplies every ciphertext in the vote vector together:

$$E(v_1) \cdot E(v_2) \cdot \dots \cdot E(v_n) = E(v_1 + v_2 + \dots + v_n)$$

and `VoteSumProof` proves that this aggregated ciphertext encrypts exactly 1:

$$v_1 + v_2 + \dots + v_n = 1$$

which is precisely the condition for exactly one proposal selected.

4.6.3 Fiat-Shamir Transform

Both proofs above rely on the Fiat-Shamir transform to become non-interactive. In a classic interactive proof, the verifier would send a random challenge to the prover and wait for a response; that back-and-forth simply does not fit the way an on-chain program works, where a proof has to stand on its own inside a single transaction.

We need to replace that random challenge with a hash of public data:

$$c = H(\text{public inputs})$$

For `VoteProof`, that hash covers the public key, the ciphertext, and the proof's commitments. For `VoteSumProof`, it covers the public key, the aggregated ciphertext, and its own commitments. During verification, the smart contract recomputes this same hash; if anything in the ciphertext or the proof has been tampered with, the hash no longer matches and the proof is rejected.

4.7 Smart Contract Design

The Anchor-based smart contract stores the election state on-chain through two main accounts: `Ballot` and `VoterRecord`.

4.7.1 Ballot Account

`Ballot` holds the global state of the election, including:

- the chairperson's public key;
- the list of proposal names;
- the ElGamal public key;
- the encrypted tally;

- the number of proposals;
- the index of the final winner.

The encrypted tally itself lives directly inside this account, as an array of ciphertexts, with one slot per proposal.

4.7.2 VoterRecord Account

Each voter gets their own `VoterRecord` account, which keeps track of whether that voter is authorized, whether they have already voted, and which proposal index they voted for. This account is derived as a Program Derived Address from the ballot address and the voter address, which guarantees that every voter has exactly one record per ballot, and rules out the possibility of one voter hijacking another voter's account.

4.7.3 Program Instructions

The contract exposes four instructions in total:

- `initialize`: creates a new ballot, stores the proposals, saves the ElGamal public key, and sets up the encrypted tally;
- `register_voter`: lets the chairperson authorize a voter by creating their `VoterRecord`;
- `cast_vote`: receives an encrypted vote vector and its proofs, verifies everything, blocks double voting, and folds the vote into the encrypted tally;
- `set_final_winner`: records the final winner once the tally has been decrypted off-chain.

`cast_vote` starts by checking that the voter is authorized and has not already voted, then verifies that the proposal index is within range, that the vote vector has the right length, and that the number of attached proofs matches the number of proposals.

Only after these checks pass does the contract move on to the cryptographic verification: first every `VoteProof`, confirming that each component really is a 0 or a 1, and then `VoteSumProof`, confirming that the vector sums to exactly one selected proposal. The voter's record and the global tally are only updated once all of this has succeeded, which matters a lot: invalid votes are rejected before they ever get the chance to influence the result.

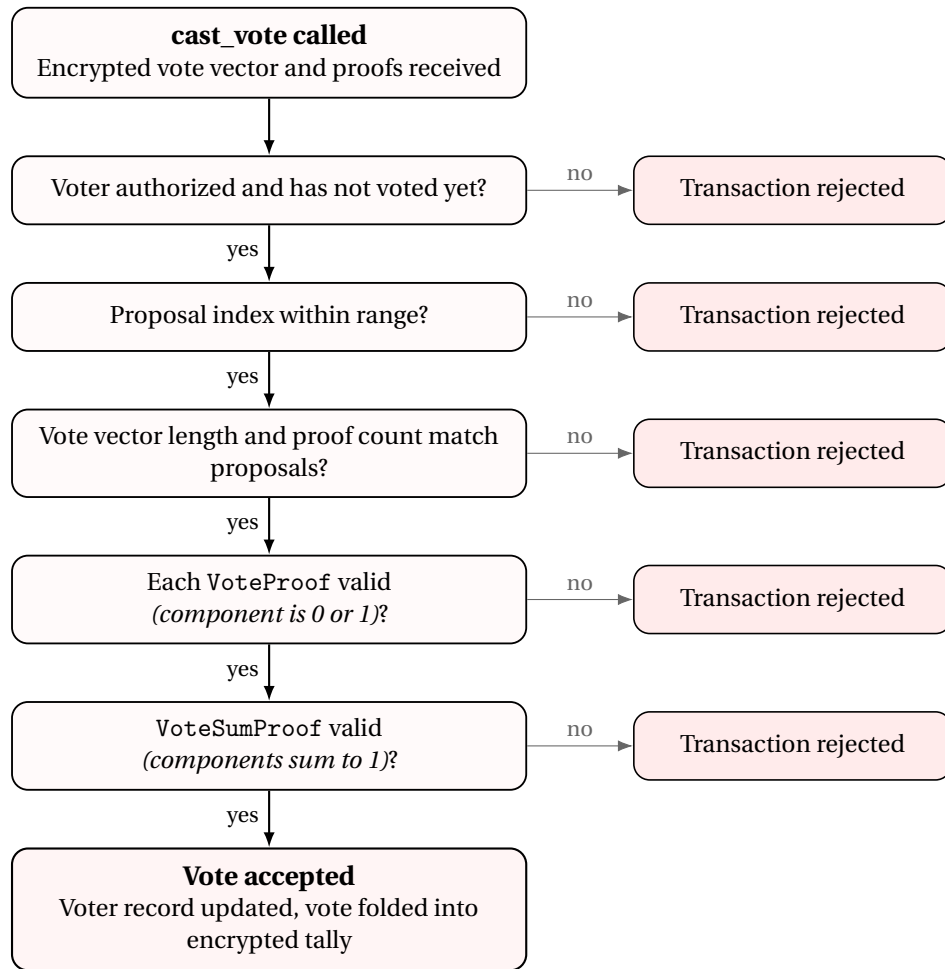


Figure 4.2: Validation flow inside the `cast_vote` instruction, showing the sequential checks that must all pass before a vote is folded into the encrypted tally.

4.8 Testing the Prototype

The prototype was tested locally with `solana_program_test`, simulating the full life of an election from start to finish: initialization, voting, and final tally decryption.

A typical local test sets up an election with three proposals, picked for being easy to remember:

- wine;
- beer;

- water.

A small ElGamal keypair is generated for the test environment, the encrypted tally starts out as one encrypted zero per proposal, and two voters are then registered and cast their votes.

The first voter votes for wine:

[1,0,0]

while the second votes for beer:

[0,1,0]

Both votes are encrypted, paired with their zero-knowledge proofs, and submitted to the contract.

4.8.1 Final Tally Decryption

Once voting wraps up, the encrypted tally is read back from the `Ballot` account and decrypted off-chain with the ElGamal secret key. Lets pretend, one vote went to wine and one to beer, resulting in a tie. To resolve ties, the chairperson manually picked the final winner via the `set_final_winner` instruction.

4.8.2 Testing Kaonashi from Scratch

To test the first version of Kaonashi, a local Solana test environment was used with `solana-program-test`. The test checks the full voting flow: creating the ballot, registering voters, casting encrypted votes, blocking invalid votes, and reading the final result.

The election is initialized with three proposals: wine, beer, and water. Two voters are created and registered, each with a voter record PDA.

```

beatrizlaranjinha@Beatrizs-Air /Applications/projects/Kaonashi2/kaonashi_version1/rust-tests$ cd /Applications/projects/Kaonashi2/kaonashi_version1/rust-tests
RUST_LOG=off BPF_OUT_DIR=./target/deploy cargo run --quiet
Initialize completed
Chairperson: BaPgPNDcCtvGrZTWVFGWUrdxfzsgEwfoEDWUa4tdMEFh
Ballot account: EBA1UGo76tReyEfhWfXLX8vhTQhiWz1FzvzhgJnpQa7Q

Voter1: 4iGmyWbFv8Pej5ZV4xbHC83q4c1Hnju5sR7twjmgRRit
Voter2: 93MCuE6C69Dhfc7AAf2wfg61TKLmTMVHzjMfwgMrvMr

Voter1 PDA: 3ztf8v1CHrAgXUSZGtpyMEYVpwri91Bt23ZyCedMLH6
Voter2 PDA: 22mPRj1heD9cWm5BpkyN2tA6e6p2RR6pVAoAdgDa2wAe

Voter1 authorized
Voter2 authorized

Voter1 voted for wine
Voter2 voted for beer
Double voting correctly blocked

```

Figure 4.3: Ballot initialization and voter registration.

After that, Voter1 votes for wine and Voter2 votes for beer. The test also tries invalid actions: double voting, a manipulated encrypted vote, and an invalid vote vector $[1, 1, 0]$. All of them are correctly blocked.

```

Malicious voter authorized
Manipulated vote correctly blocked

Double-choice voter authorized
Vote [1,1,0] was blocked

Final election state:
Chairperson: BaPgPNDcCtvGrZTWVFGWUrdxfzsgEwfoEDWUa4tdMEFh
Proposal count: 3
Final winner index: 255

Encrypted tally:
0 → wine: Ciphertext { c1: 62, c2: 76 }
1 → beer: Ciphertext { c1: 127, c2: 11 }
2 → water: Ciphertext { c1: 112, c2: 124 }

Decrypted results:
0 → wine votes: 1
1 → beer votes: 1
2 → water votes: 0

Chairperson set the final winner: beer

```

Figure 4.4: Valid votes and blocked invalid voting attempts.

The final tally remains encrypted on-chain and is decrypted off-chain. The result is one vote for wine, one vote for beer, and zero votes for water. Since there is a tie, the chairperson selects beer as the final winner.

```
Voter1 record:
can_vote: true
has_voted: true
vote: 0

Voter2 record:
can_vote: true
has_voted: true
vote: 1
Updated final winner index: 1
Final winner: beer
book of leonard shen @ 2021-10-15 11:15:15 (AnJianShi/kaonashi/kaonashi-version1/curt-knots)
```

Figure 4.5: Final voter records and final winner.

These tests show that Kaonashi from Scratch accepts valid encrypted votes, rejects invalid ones, keeps the tally encrypted, and produces the correct final result.

4.9 Conclusion

Building Kaonashi from scratch turned out to be a genuinely useful first step, not because the result was production-ready, but because it proved that the core building blocks of an encrypted voting system could actually be implemented and tested on Solana.

This prototype showed that votes can be represented as one-hot vectors, encrypted with ElGamal, aggregated homomorphically and validated through zero-knowledge proofs. It also showed that the blockchain itself can hold the full election state, voter records and encrypted tally included, while reliably rejecting double voting, manipulated ciphertexts, and malformed vote vectors.

That is exactly why this chapter should be read as a foundation: it delivered a working cryptographic prototype and it made clear which parts of the design were worth keeping, which needed improvement, and which had to be replaced in the next version.

Chapter

5

Kaonashi Scales Up

5.1 Introduction

The first version of the system was deliberately built from scratch. Before trusting any abstraction, the goal was to understand what was actually happening behind the scenes: ElGamal encryption, homomorphic aggregation, vote encoding, and zero-knowledge proofs.

This approach showed how a vote is defined as a one-hot vector, how each position is encrypted independently, how encrypted votes are combined into an encrypted tally, and how only the final aggregated result is eventually revealed. It also made clear why zero-knowledge proofs are not optional. Without them, a voter could submit an encrypted but completely invalid value. The vote would remain private, but the election logic would no longer be trustworthy.

However, manual cryptography has an obvious drawback. It is useful for learning and prototyping, but it is not the best approach for a system that aims to move closer to a production-like environment. For this reason, the next version of Kaonashi *grows up* by replacing handcrafted cryptographic components with established primitives from the Solana ecosystem, namely `solana_zk_sdk`, Curve25519/Ristretto-based operations, and the Dalek library family.

The main difference is that the system scales by using a RC. This allows it to handle a large number of simultaneous votes by batching them into a single transaction that is recorded on Solana, the layer-1 blockchain ledger.

5.1.1 Voting Flow

Lets now explain how the flow works, from the moment the user votes until the vote is ready to be included in a batch.

It starts simply enough: the user picks a movie. Behind the scenes, that choice becomes a one-hot vector. Say the ballot has four proposals; voting for the third one looks like this:

$$[0, 0, 1, 0]$$

Before encrypting anything, the frontend fetches the ElGamal public key tied to the selected decade from the backend. Everything that follows happens locally, using that key.

Each position of the vector is encrypted on its own, using Twisted ElGamal over Ristretto-compatible group elements, with a new random opening per position (kept as a `PedersenOpening` and never sent to the backend). So the plaintext vector:

$$[0, 0, 1, 0]$$

turns into an encrypted one:

$$[0, 0, 1, 0] \longrightarrow [C_0, C_1, C_2, C_3]$$

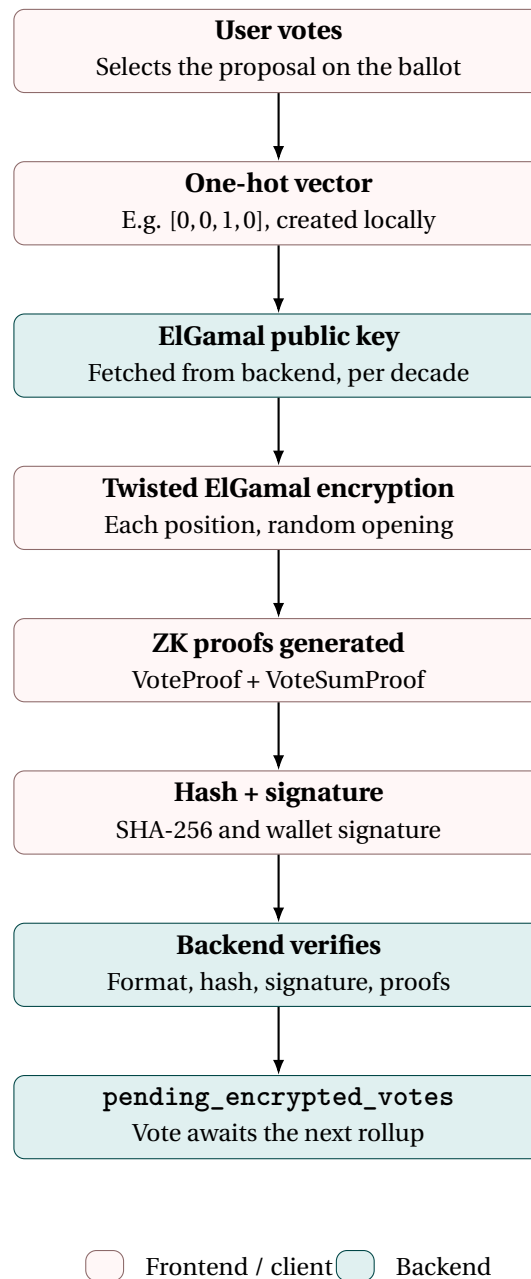


Figure 5.1: Encrypted voting flow, before the rollup.

At this point the backend has four ciphertexts and no idea which one hides the actual vote, which is the whole point. However the backend also can't tell a valid vote from a malformed one just by looking. So before submitting anything, the frontend attaches zero-knowledge proofs that vouch for the vector's

shape without revealing its contents, the details of which are unpacked in the next section.

Once the encrypted vote and its proofs are ready, the frontend hashes the whole submission with SHA-256. This hash doubles as the vote's receipt identifier:

$$\text{encrypted_vote_hash} = H(\text{encrypted_vote} \parallel \text{metadata})$$

and the submission is then signed with the voter's wallet secret key, which never leaves the frontend. The backend only ever sees the resulting signature, checked later against the voter's public key.

The backend runs through its checklist: format looks right, the hash matches, the signature checks out, and both proofs hold up. Pass all four, and the encrypted vote is stored in:

pending_encrypted_votes

where it sits until the next rollout batch picks it up.

5.1.2 Toy Example for the Voting Flow

To make the process easier to understand, consider a simplified ballot with four films:

Index	Film
0	Interstellar
1	Batman
2	Matrix
3	Fight Club

If the user votes for *Matrix*, the selected index is 2, and the vote vector is:

$$v = [0, 0, 1, 0]$$

In the real implementation, the vote is encrypted using Twisted ElGamal over Ristretto-compatible group elements. For explanation purposes, we can represent each encrypted value as:

$$C_i = \text{Enc}(v_i; r_i)$$

where v_i is the vote value at position i , and r_i is the random opening used during encryption.

For the example vote:

$$[0, 0, 1, 0]$$

the frontend encrypts each position separately:

$$C_0 = \text{Enc}(0; r_0)$$

$$C_1 = \text{Enc}(0; r_1)$$

$$C_2 = \text{Enc}(1; r_2)$$

$$C_3 = \text{Enc}(0; r_3)$$

This produces the encrypted vote vector:

$$[C_0, C_1, C_2, C_3]$$

Even though three positions encrypt the value 0, the ciphertexts are different because each one uses a different random opening. This prevents the backend from learning which positions contain 0 or 1.

5.1.2.1 VoteProof

The first type of proof is the *VoteProof*. A *VoteProof* is generated for each ciphertext in the encrypted vote vector.

Its purpose is simple: it proves that each encrypted component represents either 0 or 1.

For a ciphertext C_i , the proof guarantees that:

$$C_i \text{ encrypts either 0 or 1}$$

without revealing which one.

This is necessary because the backend cannot decrypt C_i . Without a proof, a malicious user could encrypt an invalid value, such as 2, 5, or any other number, and the backend would not be able to detect this from the ciphertext alone.

Conceptually, the proof is an OR-proof. The prover shows that one of the following statements is true:

$$C_i = \text{Enc}(0; r_i)$$

or:

$$C_i = \text{Enc}(1; r_i)$$

without revealing which statement is the real one. This means that the backend learns only that the ciphertext is binary. It does not learn whether the encrypted value is 0 or 1.

For the example vote:

$$[0, 0, 1, 0]$$

the frontend generates one proof per ciphertext:

$$C_0 \rightarrow \text{VoteProof}_0$$

$$C_1 \rightarrow \text{VoteProof}_1$$

$$C_2 \rightarrow \text{VoteProof}_2$$

$$C_3 \rightarrow \text{VoteProof}_3$$

The backend verifies each proof using only public information:

$$\text{ElGamal public key}, \quad C_i, \quad \text{VoteProof}_i$$

The backend does not receive:

$$v_i$$

and it does not receive the secret opening as evidence of the plaintext vote:

$$r_i$$

Therefore, the backend can verify that every encrypted component is valid, while still learning nothing about which component contains the selected movie.

However, this is not enough to prove that the whole vote is valid. For example, the vector:

$$[1, 1, 0, 0]$$

also contains only binary values, but it is not a valid vote because it selects two proposals. Therefore, another proof is needed.

5.1.2.2 VoteSumProof

The second type of proof is the *VoteSumProof*. This proof guarantees that the encrypted components sum to exactly 1.

The desired condition is:

$$\sum_i v_i = 1$$

For the example vote:

$$[0, 0, 1, 0]$$

this is true because:

$$0 + 0 + 1 + 0 = 1$$

However, the backend cannot see the plaintext values. It only sees the encrypted ciphertexts:

$$[C_0, C_1, C_2, C_3]$$

This is where the homomorphic property of Twisted ElGamal is used. Since the encryption is additively homomorphic, ciphertexts can be added together without decrypting them:

$$C_{\text{sum}} = C_0 + C_1 + C_2 + C_3$$

If the encrypted vote is valid, then C_{sum} should be an encryption of 1:

$$C_{\text{sum}} = \text{Enc}(1; r_{\text{sum}})$$

where:

$$r_{\text{sum}} = r_0 + r_1 + r_2 + r_3$$

The frontend then provides a zero-knowledge proof that this encrypted aggregate corresponds to the value 1, without revealing the individual vote values and without revealing which ciphertext contains the selected proposal.

Another way to understand this is to imagine that the verifier checks whether the encrypted sum, after subtracting an encryption of 1, becomes an encryption of zero:

$$C_{\text{sum}} - \text{Enc}(1; r_{\text{sum}}) = \text{Enc}(0; 0)$$

The proof convinces the backend that this relationship is correct, without exposing the plaintext vote.

Therefore, the *VoteSumProof* prevents invalid votes such as:

$$[1, 1, 0, 0]$$

because their sum is 2, not 1. It also prevents votes such as:

$$[0, 0, 0, 0]$$

because their sum is 0, not 1.

5.1.2.3 Why Both Proofs Are Needed

The two proof types are complementary.

The *VoteProofs* prove that each individual encrypted component is binary:

$$v_i \in \{0, 1\}$$

The *VoteSumProof* proves that the encrypted components sum to exactly one:

$$\sum_i v_i = 1$$

Together, they prove that the encrypted vote is a valid one-hot vector:

$$v = [0, 0, 1, 0]$$

without revealing the position of the selected proposal.

This gives the backend exactly the information it needs:

the vote is valid

and nothing more.

It does not learn:

which movie was selected

This is the role of zero-knowledge in the voting flow: it allows the backend to enforce election rules while preserving vote privacy.

5.1.2.4 Backend Verification

When the backend receives the encrypted vote submission, it performs the following checks:

1. It verifies that the encrypted vote has the correct format.
2. It recomputes the SHA-256 hash of the encrypted submission and compares it with the received hash.
3. It verifies the wallet signature over the signed message.
4. It verifies the proof data showing that each encrypted component is binary.
5. It verifies the proof data showing that the encrypted vector sums to exactly one.

Only after these checks succeed is the encrypted vote stored in:

`pending_encrypted_votes`

This means that the backend accepts only votes that are encrypted, correctly signed, and proven to be valid one-hot votes.

5.1.3 Rollup

As we have just seen, after the backend accepts an encrypted vote, the vote is not immediately submitted on-chain by itself. Instead, it is temporarily stored in:

`pending_encrypted_votes`

This is where the rollup mechanism enters.

The purpose of the rollup is to group multiple encrypted votes into a batch, compute a compact cryptographic summary of that batch, and submit only the necessary aggregated information to Solana.

This process is divided into two parts:

- the off-chain part, handled by the backend API;
- the on-chain part, handled by the Solana smart contract.

5.1.3.1 Off-chain Rollup Coordination

The backend does not know the plaintext vote. It never sees anything like:

“this user voted for movie X”

Instead, it sees only encrypted votes, vote hashes, signatures, and proof data.

A batch is a group of accepted encrypted votes. For example:

$\text{Batch}_1 = \{\text{vote}_A, \text{vote}_B, \text{vote}_C, \text{vote}_D\}$

Each encrypted vote submission is represented as a leaf in a Merkle tree:

$\text{vote}_A \rightarrow L_0; \text{vote}_B \rightarrow L_1; \text{vote}_C \rightarrow L_2; \text{vote}_D \rightarrow L_3$

In this implementation, the leaf is not just a random identifier. Conceptually, it commits to information about the encrypted submission, such as the wallet identifier, the public key, the decade identifier, the encrypted vote hash, and the encrypted vote bytes.

A simplified representation of a leaf is:

$$L_i = H(\text{wallet_id} \parallel \text{public_key} \parallel \text{decade_id} \parallel \text{encrypted_vote_hash} \parallel \text{encrypted_vote})$$

This means that the Merkle tree commits to the encrypted vote submission, not to the plaintext vote. Therefore, the receipt can later be used to check whether a specific encrypted submission was included in a batch.

For four votes, the Merkle tree can be represented as follows:

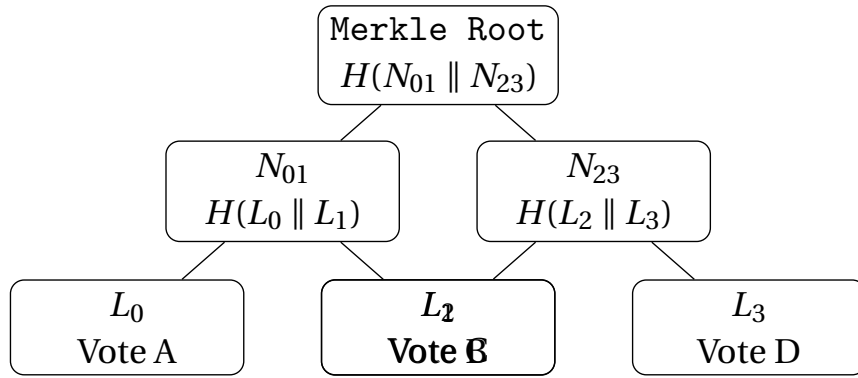


Figure 5.2: Merkle tree for a batch with four encrypted vote submissions.

The internal nodes are computed as:

$$N_{01} = H(L_0 \parallel L_1)$$

$$N_{23} = H(L_2 \parallel L_3)$$

and the Merkle root is:

$$\text{merkle_root} = H(N_{01} \parallel N_{23})$$

The Merkle root is a compact cryptographic commitment to all encrypted submissions inside the batch.

However, the Merkle tree only solves the inclusion problem. It tells us which encrypted submissions were included in the batch, but it does not count the votes.

For counting, the backend computes the homomorphic sum of the encrypted votes in the batch.

For example, assume a simplified encrypted batch containing three-proposal votes:

$$Enc([1,0,0]), \quad Enc([0,1,0]), \quad Enc([1,0,0]), \quad Enc([0,0,1])$$

Because ElGamal encryption is additively homomorphic, the backend can add the encrypted votes directly:

$$Enc([1,0,0]) + Enc([0,1,0]) + Enc([1,0,0]) + Enc([0,0,1]) = Enc([2,1,1])$$

The result is called the encrypted batch tally:

$$\text{encrypted_batch_tally} = Enc([2,1,1])$$

This means that, inside this batch, movie 0 has two votes, movie 1 has one vote, and movie 2 has one vote.

After the off-chain rollup step, the backend has a batch containing:

`batch = {batch_id, decade_id, vote_count, merkle_root, encrypted_batch_tally, receipt}`

5.1.3.2 On-chain Rollup State

The API does not submit every individual encrypted vote to Solana. Instead, it submits only a compact rollup summary:

`merkle_root, encrypted_batch_tally, batch_size`

Solana receives this summary and updates the on-chain ballot state.

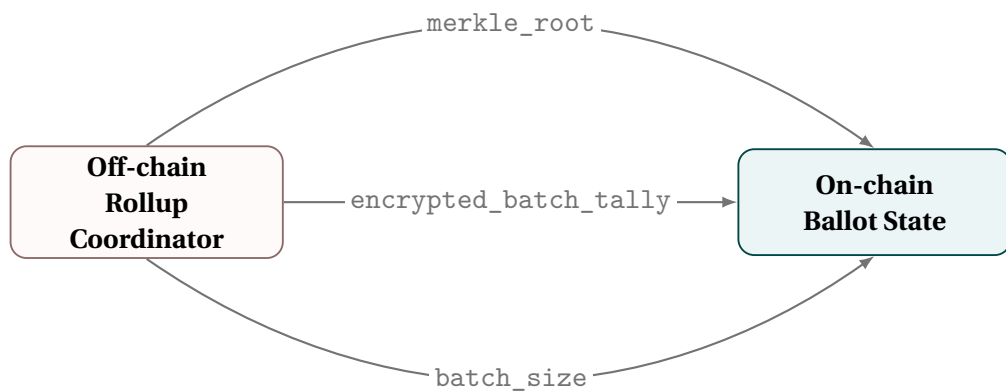


Figure 5.3: Compact rollup batch summary submitted from the API to the on-chain ballot.

The simplified ballot structure below focuses on the fields most relevant to the rollup state:

```
pub struct Ballot {
    pub encrypted_tally: Vec<[u8; 64]>,
    pub merkle_root:    [u8; 32],
    pub total_votes:    u64,
    pub batch_count:    u64,
}
```

Rather than storing every individual vote, receipt, or Merkle proof, the Ballot account keeps only a compact summary of the election state.

Batches are submitted through the `submit_rollup_batch` instruction, which receives the new Merkle root, the encrypted tally for that batch, and the number of votes it contains:

```
pub fn submit_rollup_batch(
    ctx:                Context,
    new_merkle_root:    [u8; 32],
    encrypted_batch_tally: Vec<[u8; 64]>,
    batch_size:         u64,
) -> Result<()>
```

Before accepting the batch, the contract checks that it is well-formed:

- `batch_size` must be greater than zero;
- `encrypted_batch_tally` must contain exactly one ciphertext per proposal;
- every ciphertext must have the expected serialized format.

Once validation passes, the contract folds the new batch tally into the global encrypted tally using ciphertext addition:

$$\text{encrypted_tally} \leftarrow \text{encrypted_tally} + \text{encrypted_batch_tally}$$

This addition is performed entirely on ciphertexts.

Finally, the remaining fields are updated to reflect the new batch:

```
merkle_root ← new_merkle_root
total_votes ← total_votes + batch_size
batch_count ← batch_count + 1
```

After this instruction completes, the on-chain state reflects that a new batch has been included, how many votes it contributed, which Merkle root

represents the latest submitted batch, and how the encrypted global tally has changed.

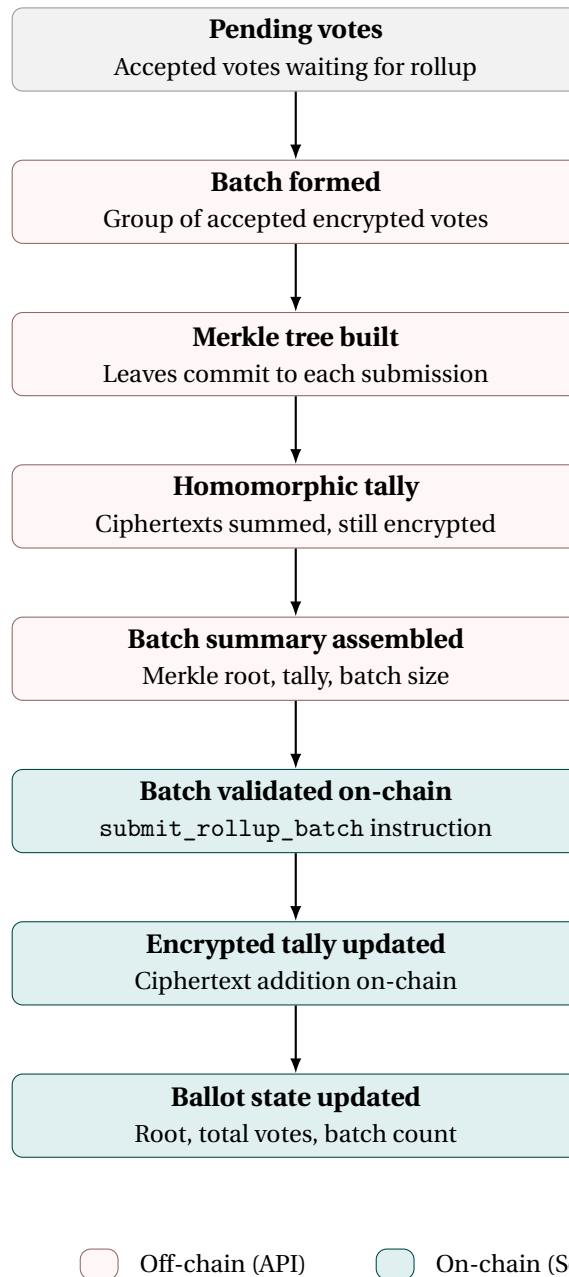


Figure 5.4: Rollup flow: batches are formed and committed off-chain, then validated and folded into the ballot state on-chain.

5.1.4 How do the Receipts Work?

Each voter receives a receipt code after submitting an encrypted vote. This receipt code is the hash of the encrypted vote submission:

```
receipt_code = encrypted_vote_hash
```

The purpose of the receipt is simple: it allows the voter to later verify that their encrypted vote was included in a submitted batch.

The receipt does not reveal the selected movie. It does not contain the plaintext vote. It only identifies the encrypted submission.

The flow is:

```
encrypted vote → hash → receipt code
```

For example, after voting, the user may receive a receipt code such as:

```
a82f9c...91e4
```

This code can later be pasted into the results page.

After pending encrypted votes are submitted as a batch, the backend associates each receipt code with batch metadata such as:

```
vote_hash, batch_id, merkle_root
```

The user can then verify the receipt by submitting the vote hash to the API. If the hash is found in the backend's recorded batch receipts, the system returns that the vote was included in a batch.

5.1.5 Chairperson and its Power over the Election

Every election needs someone to run the show, and in Kaonashi that role belongs to the chairperson. Unlike a normal voter, who only casts encrypted votes, the chairperson is in charge of the election lifecycle itself.

There is nothing special about how this role is set up. The chairperson is simply a Solana wallet, with the usual public/private keypair, designated as the administrator when the API starts up:

```
CHAIRPERSON_PUBLIC_KEY=7Cpzofh7d9nSMMeuhg9Fu2sWcwKhNKprrsWa7pHemMk
```

The frontend has no opinion on who the chairperson is; it simply asks the backend whether a connecting wallet matches this configured key. If it does, that user is redirected to the chairperson page, where the administrative actions live.

Keeping the private key safe. The private key never touches the source code. Locally, it lives in the Solana keypair file at `~/.config/solana/id.json`, and to sign administrative actions from the browser it is converted to Base58 and pasted manually into the chairperson page. From there, it is held only in memory for the duration of the session, via a simple reactive signal:

```
let chairperson_secret_key = RwSignal::new(String::new());
```

This key is used purely to sign administrative messages locally, such as:

```
Kaonashi admin action .. public_key: ... action: create_ballots
```

Once signed, the frontend forwards only the public key, the message, and the resulting signature to the backend. The same applies on the voter side: voters sign their encrypted vote submissions locally and only ever send the public key, the encrypted vote, the proof data, the vote hash, and the signature. In both cases, the private key itself never leaves the browser, and the backend authenticates actions purely through signature verification.

Chairperson actions. With this authorization model in place, the chairperson can:

- create the on-chain ballots;
- close the election to new votes;
- submit pending encrypted votes as rollup batches;
- decrypt the encrypted tally and detect ties;
- resolve ties, when necessary;
- finalize the winners on-chain.

Every one of these is gated by the same check: the backend accepts an action only if the public key matches the configured chairperson key *and* the signature is valid for that specific message. Possessing the public key alone is never enough.

For example, the message behind a `create_ballots` action looks like this:

```
Kaonashi admin action
public_key: 7Cpzofh7d9nSMMeuhg9Fu2sWcwiKhNKprrsWa7pHemMk
action: create_ballots
```

and the frontend simply forwards the trio `public_key`, `message`, `signature` for the backend to verify.

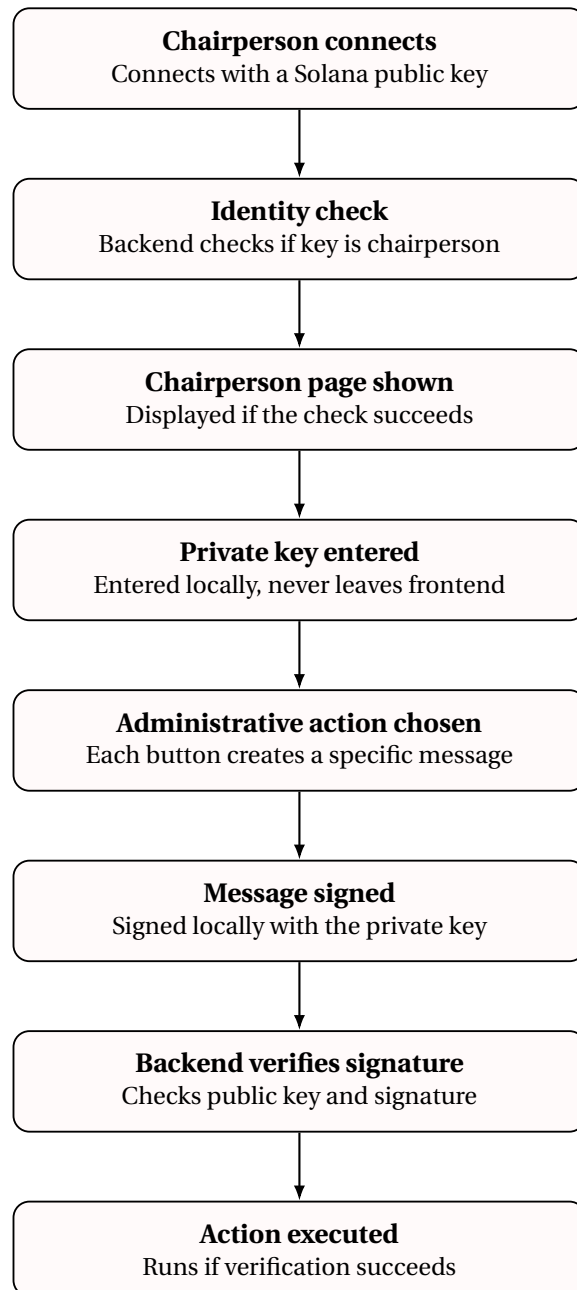


Figure 5.5: Chairperson authorization flow in Kaonashi.

In short, the chairperson has real power over how the election unfolds,

but none over what voters chose. Votes stay encrypted throughout, and the chairperson's authority is confined to process: closing the election, batching, decrypting the final tally, breaking ties, and finalizing winners, never looking at an individual plaintext vote along the way.

5.2 Watching Kaonashi on Solana Explorer

For this chapter i simulated a full election run and followed the transactions in Solana Explorer.

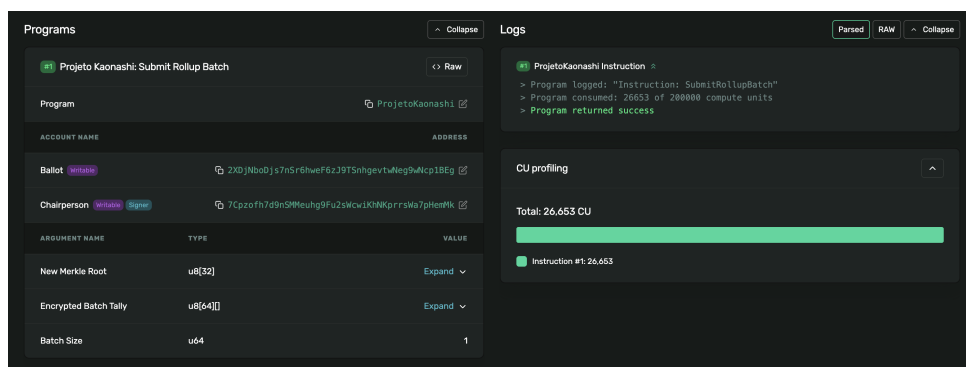


Figure 5.6: Transactions on Solana

First, the chairperson created the ballots. In the Explorer, this appears as the `Initialize` instruction. For each decade, the program creates a ballot account with the movie proposals, the ElGamal public key, and an initial encrypted tally. At this point, the election exists on-chain, but the tally is still completely empty.

Then, I submitted one encrypted vote for decade 0. The API verified the vote signature, checked the encrypted vote hash, and validated the zero-knowledge proofs. This is an important detail: the individual vote is not immediately written to the blockchain. Instead, it stays as a pending encrypted vote until it is included in a batch.

After that, the pending vote was submitted through the rollup flow. In Solana Explorer, this appears as the `SubmitRollupBatch` instruction. The transaction included a new Merkle root, an encrypted batch tally, and a batch size of 1.

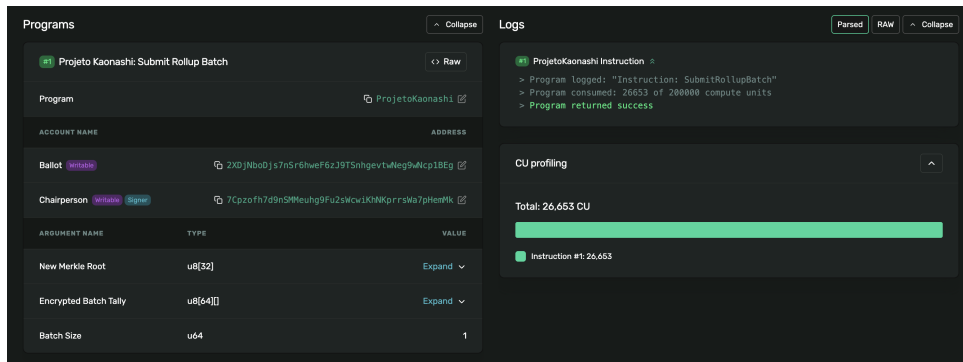


Figure 5.7: Rollup batch submission for decade 0, containing the new Merkle root, encrypted batch tally, and batch size.

The transaction fee was also very small, only 0.000005 SOL.

Next, the election was closed with the `CloseElection` instruction. The Explorer shows that the ballot account was writable and that the chairperson signed the transaction. After this step, the API no longer accepts new votes for that election.

Finally, the election was finalized. The API fetched the encrypted tally from the blockchain and decrypted it locally. For decade 0, the final result was:

[1, 0, 0, 0, 0, 0, 0, 0]

This means that the winning movie for decade 0 was *Taxi Driver*. The other ballots had no votes, which the API also reported. The final winner was then stored on-chain using the `SetFinalWinner` instruction.

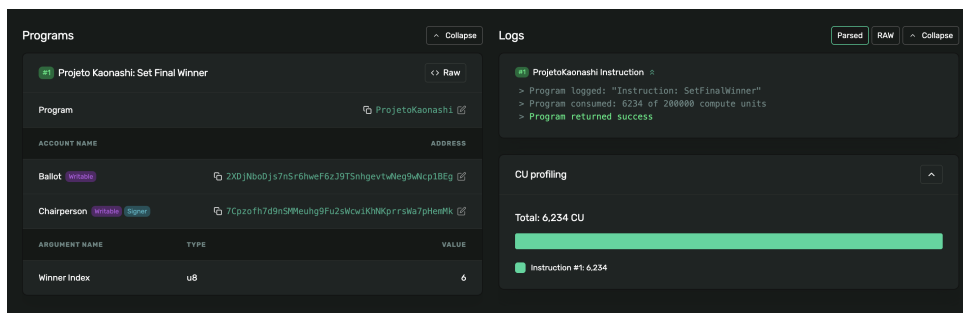


Figure 5.8: Final winner being stored on-chain after the encrypted tally was decrypted and finalized.

Overall, this test confirmed the expected behaviour: votes are encrypted and verified before being batched, batches update the on-chain encrypted tally, and the final winner is only stored after the election is closed and finalized. The blockchain never receives plaintext votes, but it still records the important election steps.

5.2.1 Testing Kaonashi Scales Up

To test *Kaonashi Scales Up*, a complete election flow was simulated through the API. In this test, the chairperson creates the ballots, ten voters submit encrypted votes, the system detects a tie, and the chairperson resolves it.

The test starts by creating the decade ballots. For this run, decade 2 was used, which corresponds to the 1990s ballot.

```
Compiling kaonashi-api v0.1.0 (/Applications/projects/kaonashi_finalProject/kaonashi-api)
Finished 'dev' profile [unoptimized + debuginfo] target(s) in 2.16s
Running 'target/debug/submit_test_vote 2'
Starting Kaonashi Scales Up test
Decade id: 2
Chairperson: 7Cpzoifh7d9nSMMeuhg9Fu2sWcwiKhNKprsrWa7pHemMk

Create ballots completed
decade 0 → ballot 785NmFyHaTWi2qhFzsrqZu9rWR97g7rmgGfFxTAVGDuU
decade 1 → ballot 55d7SHegVfedKs1EsaQ10QdASaBxGrEZ2vrrqVBGVqBBU
decade 2 → ballot 9rt381mZC2qUodUSM59jP6EPBBxXDn1WjaBhzyjHYR8D
decade 3 → ballot 8PBxGAmzWmj7WFGkdFX4nAV94hXS4xRv3jNvU5omPNLY
decade 4 → ballot 3W3nugrVW3Wmt887qhwGb9Hx7x8rhU5SovJtgeXzWuVw
decade 5 → ballot 2gsQHTBJwkj6SG9vPZAXqjfGKdzSbSPw4nUyCaZ3Hnt1
```

Figure 5.9: Chairperson creates the decade ballots.

Then, ten encrypted votes are submitted. The votes were planned to create a tie: movie 0 receives three votes, movie 1 receives three votes, and movies 2 and 3 receive two votes each. The last vote also triggers the batch submission, showing that the pending votes are grouped into a rollout batch.

```
Submitting 10 encrypted votes
Expected tie: movie 0 = 3 votes, movie 1 = 3 votes
Vote 1 → voter_01 voted for movie 0 | accepted=true, pending_votes=1, batch_submitted=false, status=Encrypted vote accepted
Vote 2 → voter_02 voted for movie 1 | accepted=true, pending_votes=2, batch_submitted=false, status=Encrypted vote accepted
Vote 3 → voter_03 voted for movie 0 | accepted=true, pending_votes=3, batch_submitted=false, status=Encrypted vote accepted
Vote 4 → voter_04 voted for movie 1 | accepted=true, pending_votes=4, batch_submitted=false, status=Encrypted vote accepted
Vote 5 → voter_05 voted for movie 2 | accepted=true, pending_votes=5, batch_submitted=false, status=Encrypted vote accepted
Vote 6 → voter_06 voted for movie 3 | accepted=true, pending_votes=6, batch_submitted=false, status=Encrypted vote accepted
Vote 7 → voter_07 voted for movie 0 | accepted=true, pending_votes=7, batch_submitted=false, status=Encrypted vote accepted
Vote 8 → voter_08 voted for movie 1 | accepted=true, pending_votes=8, batch_submitted=false, status=Encrypted vote accepted
Vote 9 → voter_09 voted for movie 2 | accepted=true, pending_votes=9, batch_submitted=false, status=Encrypted vote accepted
Vote 10 → voter_10 voted for movie 3 | accepted=true, pending_votes=0, batch_submitted=true, status=Encrypted vote accepted
```

Figure 5.10: Submission of ten encrypted votes.

After vote submission, the election is closed and the remaining batch is flushed. The first finalization detects a tie in decade 2, with tied indices [0, 1].

```
Expected result before tie resolution:
movie 0 → 3 votes
movie 1 → 3 votes
movie 2 → 2 votes
movie 3 → 2 votes

Close election completed
{"success":true,"results":[{"decade_id":0,"success":true,"status":"Closed"},{"decade_id":1,"success":true,"status":"Closed"},{"decade_id":2,"success":true,"status":"Closed"},{"decade_id":3,"success":true,"status":"Closed"},{"decade_id":4,"success":true,"status":"Closed"},{"decade_id":5,"success":true,"status":"Closed"}],"status":"Election closed across 6 decade ballot(s)."}

Flush batch completed
{"success":false,"decade_id":2,"batch_id":"","merkle_root":"","vote_count":0,"encrypted_batch_tally":[],"receipts":[],"status":"No pending encrypted votes"}

Finalize election completed
{"success":false,"results":[{"decade_id":0,"success":false,"status":"NoVotes"},{"decade_id":1,"success":false,"status":"NoVotes"},{"decade_id":2,"success":false,"status":"Tie"},{"decade_id":3,"success":false,"status":"NoVotes"},{"decade_id":4,"success":false,"status":"NoVotes"},{"decade_id":5,"success":false,"status":"NoVotes"}],"status":"0 decade ballot(s) finalized. 1 tie(s) require resolution. 5 decade ballot(s) had no votes. 0 error(s)."}

```

Figure 5.11: Election closing, batch flushing, and tie detection.

The results show that *Fight Club* and *Pulp Fiction* both received three votes. Since there is a tie, no final winner is selected automatically. The chairperson then resolves the tie by choosing movie index 1, corresponding to *Pulp Fiction*.

```
Election results:
{"decade_id":2,"decade":"1990s","ballot_address":"9rt381mZC2qUodUSM59jP6EPBBxXDn1WjaBhzyjHYR8D","total_votes":10,"winner_index":null,"winner":null,"tie_indices":[0,1],"final_winner_index":null,"final_winner":null,"results":[{"index":0,"title":"Fight Club","votes":3}, {"index":1,"title":"Pulp Fiction","votes":3}, {"index":2,"title":"Se7en","votes":2}, {"index":3,"title":"Goodfellas","votes":2}, {"index":4,"title":"Eyes Wide Shut","votes":0}, {"index":5,"title":"Casino","votes":0}, {"index":6,"title":"Fallen Angels","votes":0}, {"index":7,"title":"Reservoir Dogs","votes":0}]}

Resolve tie completed
{"success":true,"decade_id":2,"winner_index":1,"status":"Tie resolved for decade 2 with winner index 1."}

Finalize election completed
{"success":true,"results":[{"decade_id":0,"success":false,"status":"NoVotes"}, {"decade_id":1,"success":false,"status":"NoVotes"}, {"decade_id":2,"success":true,"status":"Finalized"}, {"decade_id":3,"success":false,"status":"NoVotes"}, {"decade_id":4,"success":false,"status":"NoVotes"}, {"decade_id":5,"success":false,"status":"NoVotes"}],"status":"1 decade ballot(s) finalized. 0 tie(s) require resolution. 5 decade ballot(s) had no votes. 0 error(s)."}

```

Figure 5.12: Tie resolution by the chairperson.

After the tie is resolved, the election is finalized. The final result confirms *Pulp Fiction* as the winner (index set to 1).

```
Election results:
{"decade_id":2,"decade":"1990s","ballot_address":"9rt381mZC2qUodUSM59jP6EPBBxXDn1WjaBhzyjHYR8D","total_votes":10,"winner_index":null,"winner":null,"tie_indices":[0,1],"final_winner_index":1,"final_winner":"Pulp Fiction","results":[{"index":0,"title":"Fight Club","votes":3}, {"index":1,"title":"Pulp Fiction","votes":3}, {"index":2,"title":"Se7en","votes":2}, {"index":3,"title":"Goodfellas","votes":2}, {"index":4,"title":"Eyes Wide Shut","votes":0}, {"index":5,"title":"Casino","votes":0}, {"index":6,"title":"Fallen Angels","votes":0}, {"index":7,"title":"Reservoir Dogs","votes":0}]}

Kaonashi Scales Up test completed
Final winner selected by chairperson: movie 1

```

Figure 5.13: Final result after tie resolution.

The API logs also confirm the internal verification steps. The encrypted vote signatures are checked, the zero-knowledge vote proofs are verified, and the rollup batch is submitted on-chain for decade 2.

```
Kaonashi API running at http://127.0.0.1:3000
decade 0 → ballot 785NmFyHaTwi2qhFzsrqZu9rWR97g7rmgGfXtAVGDuU
decade 1 → ballot 55d7SEgVfedKs1EsaQiQQdASaBxGrEZ2vrqVBGVqBBU
decade 2 → ballot 9rt381mZC2qUodUSM59jP6EPBBxXDn1WjaBhzyjHYR8D
decade 3 → ballot BPBxGAmzWmj7WFGkdfX4nAV94hXS4xRv3jNvU5omPNLy
decade 4 → ballot 3W3nugrVV3WMT887qhwGb9Hx7x8rhU5SovJtgeXzWuVw
decade 5 → ballot 2gsQHTBJwkj6SG9vPZAXqjFGKdzSbSPw4nUyCaZ3Hnt1
Encrypted vote signature verified
Encrypted vote hash: 3e3a4494560730fd01365631f045614c79eebdfcbebbffff4e67a21f6a5559a
Received vote proofs: 8
Encrypted vote proofs verified
Encrypted vote signature verified
Encrypted vote hash: 9821cdf9dd14a978fd5488bae978e1c6342c5b64660c3590ccda9b60d2766e72
Received vote proofs: 8
Encrypted vote proofs verified
Encrypted vote signature verified
Encrypted vote hash: 5c8a73ee2b0043bb1fa069e8b6376061036dc92395185a5e3ff0511aa31085b0
Received vote proofs: 8
Encrypted vote proofs verified
Encrypted vote signature verified
Encrypted vote hash: 67a5000b59dae8c6c1a047283d7758a5cff2859dcab5de7606190d0af48576ed
```

Figure 5.14: API logs showing vote proof verification and rollup batch submission.

The API also shows the on-chain closing of the decade ballots, the tie detection, and the finalization after the chairperson chooses the winner.

```
Encrypted vote proofs verified
Submitted rollup batch on-chain for decade 2. Ballot: 9rt381mZC2qUodUSM59jP6EPBBxXDn1WjaBhzyjHYR8D. Batch size: 10
Election closed on-chain for decade 0
Election closed on-chain for decade 1
Election closed on-chain for decade 2
Election closed on-chain for decade 3
Election closed on-chain for decade 4
Election closed on-chain for decade 5
Election has no votes for decade 0. Results: [0, 0, 0, 0, 0, 0, 0, 0]
Election has no votes for decade 1. Results: [0, 0, 0, 0, 0, 0, 0, 0]
Tie detected for decade 2. Tied indices: [0, 1]. Results: [3, 3, 2, 2, 0, 0, 0, 0]
Election has no votes for decade 3. Results: [0, 0, 0, 0, 0, 0, 0, 0]
Election has no votes for decade 4. Results: [0, 0, 0, 0, 0, 0, 0, 0]
Election has no votes for decade 5. Results: [0, 0, 0, 0, 0, 0, 0, 0]
Election has no votes for decade 0. Results: [0, 0, 0, 0, 0, 0, 0, 0]
Election has no votes for decade 1. Results: [0, 0, 0, 0, 0, 0, 0, 0]
Election finalized for decade 2. Winner: Pulp Fiction. Results: [3, 3, 2, 2, 0, 0, 0, 0]
Election has no votes for decade 3. Results: [0, 0, 0, 0, 0, 0, 0, 0]
Election has no votes for decade 4. Results: [0, 0, 0, 0, 0, 0, 0, 0]
Election has no votes for decade 5. Results: [0, 0, 0, 0, 0, 0, 0, 0]
```

Figure 5.15: API logs showing tie detection and finalization.

These tests show that *Kaonashi Scales Up* can accept encrypted votes, verify signatures and zero-knowledge proofs, submit a rollup batch on-chain, detect ties, and allow the chairperson to select the final winner.

5.3 Frontend Interface

The improved version of Kaonashi also includes a frontend interface. The mainpages are the home page, which introduces the system, the ballot page, where the voter selects and submits a vote, and the election panel, where the chairperson manages the election steps.

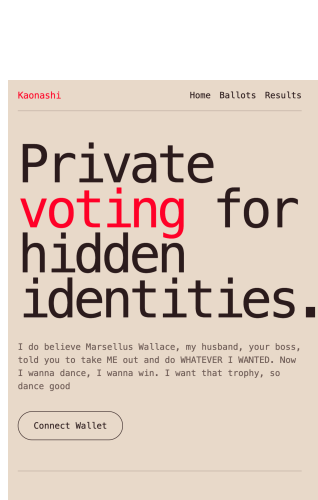


Figure 5.16: *

Home page

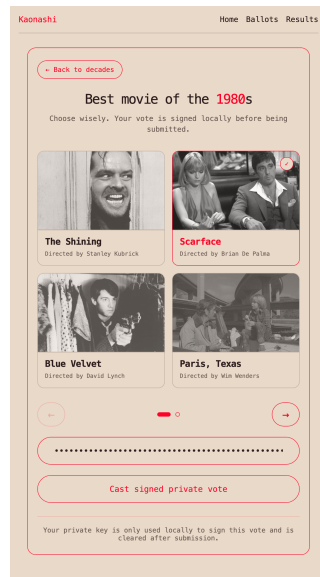


Figure 5.17: *

Ballot page

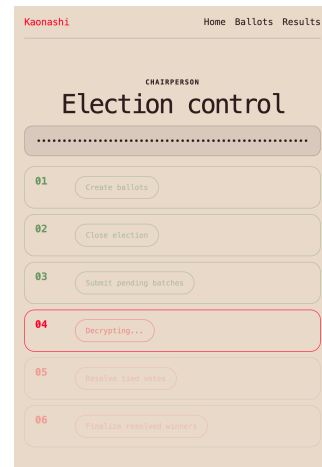


Figure 5.18: *

Election panel

Figure 5.19: Main frontend pages of the improved Kaonashi system.

5.4 Conclusion

This chapter showed how Kaonashi evolved from a handcrafted prototype into a more scalable system using Solana cryptographic primitives. Votes are encrypted with ElGamal, verified through zero-knowledge proofs, and grouped into rollup batches before being submitted on-chain. The chairperson manages the election lifecycle through wallet signatures, without accessing individual votes. The tests, including a simulated ten-vote election with a tie, confirmed that votes remain private, batches are correctly processed, and the final winner is only selected after closing and finalization. Overall, Kaonashi Scales Up preserves the privacy and correctness of the first version while improving scalability.

Chapter

6

Conclusions and Future Work

6.1 Main Conclusions

This project showed that a privacy-preserving voting system combining homomorphic encryption, zero-knowledge proofs, and a Solana smart contract is feasible at prototype scale. The first version demonstrated that votes can be validated on-chain without ever being decrypted, while the second version showed that this approach can be scaled efficiently using established cryptographic libraries and a rollup architecture, confirmed by an end-to-end test on Solana Explorer with very low transaction costs.

6.2 Future Work

This project focused mainly on understanding blockchain, advanced cryptographic techniques, zero-knowledge proofs, Rust, and Solana.

As future work, Kaonashi could be tested with more voters and larger elections to better evaluate scalability. Another possible direction would be to study real-world e-election systems and compare their challenges with the approach followed in Kaonashi, although this was outside the scope of this project.

Bibliography

- [1] A. Yakovenko, “Solana: A new architecture for a high performance blockchain,” 2018, accessed: 2026-03-17. [Online]. Available: <https://solana.com/solana-whitepaper.pdf>
- [2] S. Nakamoto, “Bitcoin: A peer-to-peer electronic cash system,” 2008.
- [3] A. Narayanan, J. Bonneau, E. Felten, A. Miller, and S. Goldfeder, *Bitcoin and Cryptocurrency Technologies: A Comprehensive Introduction*. Princeton University Press, 2016. [Online]. Available: https://www.lop.net/pdf/princeton_bitcoin_book.pdf
- [4] C. University, “What is solana’s proof of history (poh)?” 2024, accessed: 2026-03-17. [Online]. Available: <https://crypto.com/pt/university/what-is-solanas-proof-of-history-sol-consensus-mechanism>
- [5] Ledger Academy, “Byzantine fault tolerance in crypto: What is it?” 2024, accessed: 2026-03-17. [Online]. Available: <https://www.ledger.com/academy/topics/blockchain/byzantine-fault-tolerance-in-crypto-what-is-it>
- [6] L. Lamport, R. Shostak, and M. Pease, “The byzantine generals problem,” *ACM Transactions on Programming Languages and Systems*, vol. 4, no. 3, pp. 382–401, 1982.
- [7] V. Mulder, A. Mermoud, V. Lenders, and B. Tellenbach, Eds., *Trends in Data Protection and Encryption Technologies*. Cham, Switzerland: Springer, 2023. [Online]. Available: <https://link.springer.com/book/10.1007/978-3-031-33386-6>
- [8] D. Boneh and V. Shoup, *A Graduate Course in Applied Cryptography*. Self-published, 2023, version 0.6. [Online]. Available: <https://toc.cryptobook.us/book.pdf>
- [9] L. Babai, “Graph isomorphism in quasipolynomial time,” *arXiv preprint arXiv:1512.03547*, 2015. [Online]. Available: <https://arxiv.org/abs/1512.03547>

- [10] J. Katz and Y. Lindell, *Introduction to Modern Cryptography*, 2nd ed. CRC Press, 2014, standard reference for modern cryptography, including El-Gamal, DDH, and zero-knowledge proofs.
- [11] D. Chaum and T. P. Pedersen, “Wallet databases with observers,” in *Advances in Cryptology — CRYPTO '92*. Berlin, Heidelberg: Springer Berlin Heidelberg, 1993, pp. 89–105.
- [12] A. Fiat and A. Shamir, “How to prove yourself: Practical solutions to identification and signature problems,” in *Advances in Cryptology — CRYPTO '86*. Berlin, Heidelberg: Springer, 1987, pp. 186–194.
- [13] Changelly Team. (2025) What are zk rollups? Accessed: 2026-03-18. [Online]. Available: <https://changelly.com/blog/what-are-zk-rollups/>
- [14] U. Feige, A. Fiat, and A. Shamir, “Zero-knowledge proofs of identity,” *J. Cryptol.*, vol. 1, no. 2, p. 77–94, Aug. 1988. [Online]. Available: <https://doi.org/10.1007/BF02351717>
- [15] T. Tashev. (2025) Optimistic vs zero-knowledge proof: Rollups compared. Accessed: 2026-03-18. [Online]. Available: <https://www.webopedia.com/crypto/learn/ethereum-rollups/>
- [16] Rust Project Developers, “The rust programming language,” <https://www.rust-lang.org/>, 2026, accessed: 2026-03-21.
- [17] Anchor Documentation. (2026) Program structure. Accessed: 2026-04-02. [Online]. Available: <https://www.anchor-lang.com/docs/basics/program-structure>
- [18] Tokio Project, “Tokio: An asynchronous rust runtime,” 2026, accessed: 2026-04-25. [Online]. Available: <https://tokio.rs/>